

UnifiedBus™ (UB) Software Reference Design for Operating Systems

Revision: 2.0

Release Date: 2025-12-31

Copyright © 2025 Huawei Technologies Co., Ltd. All Rights Reserved.

Your use of this document is subject to the terms and conditions of the Creative Commons Attribution 4.0 International Public License (hereinafter referred to as "CC BY 4.0 License"). To view the full content of this license, visit <https://creativecommons.org/licenses/by/4.0/legalcode.txt>.

By using, reproducing, modifying, distributing, or displaying any part of this document, you hereby accept and agree to be bound by the terms and conditions of the CC BY 4.0 License.

UnifiedBus™ is a trademark of Huawei. All other trademarks, trade names, product names, service names, and company names mentioned or displayed in this document are the property of their respective owners.

Contents

1 Introduction.....	6
1.1 Purpose	6
1.2 Scope.....	6
1.3 References	7
2 Architecture.....	8
3 Device Mgmt.....	11
3.1 Overview.....	11
3.2 Functional Architecture	12
3.3 Usage Process	16
3.4 Maintenance and Debugging.....	17
4 Memory Mgmt	18
4.1 Overview.....	18
4.2 Functional Architecture	18
4.3 Usage Process	20
4.4 Maintenance and Debugging.....	24
4.5 UBTurbo	24
5 Communication.....	28
5.1 Overview.....	28
5.2 Functional Architecture	28
5.3 URMA	29
5.4 URPC.....	35
5.5 UMS.....	39
6 Virtualization	41
6.1 Overview.....	41
6.2 Functional Architecture	43
6.3 Usage Process	44
6.4 Maintenance and Debugging.....	46
7 RAS	49
7.1 Overview.....	49
7.2 Functional Architecture	50
7.3 Fault Detection	51
7.4 Fault Isolation and Recovery	56
Appendix A Abbreviations	59

Figures

Figure 1-1 UB system diagram	6
Figure 2-1 Software architecture of UB OS Component	8
Figure 3-1 UB device management architecture	12
Figure 3-2 UB device driver model	14
Figure 3-3 UB device tree	15
Figure 3-4 UB device service model.....	15
Figure 3-5 UB service device tree	16
Figure 4-1 UB Memory Management (UBMM) software architecture	18
Figure 4-2 Memory borrowing process	20
Figure 4-3 OWNERSHIP state transitions	21
Figure 4-4 Memory sharing process.....	22
Figure 4-5 Allocating Home-side memory to a device	23
Figure 4-6 Creating a UB memory descriptor in user space.....	24
Figure 4-7 UBTurbo module architecture	25
Figure 4-8 Hot and cold data identification and migration	26
Figure 4-9 Adjusting the remote-to-local memory ratio.....	27
Figure 5-1 UMDK functional architecture.....	28
Figure 5-2 URMA module architecture	30
Figure 5-3 URMA usage process.....	31
Figure 5-4 Jetty management.....	31
Figure 5-5 Segment usage process.....	32
Figure 5-6 URPC module architecture.....	36
Figure 5-7 URPC usage process	37
Figure 5-8 Channel logic.....	38
Figure 5-9 UMS module architecture.....	39
Figure 6-1 Mainstream device virtualization technologies.....	41
Figure 6-2 NIC traffic diagram in traditional architectures	41
Figure 6-3 UB device virtualization	42
Figure 6-4 NIC traffic diagram in UB.....	43
Figure 6-5 UB virtualization architecture.....	43
Figure 7-1 UB RAS framework	50
Figure 7-2 UB device fault detection framework.....	52
Figure 7-3 Remote memory fault detection framework	53
Figure 7-4 Communication fault reporting process.....	54
Figure 7-5 OOM prevention processing policy	55
Figure 7-6 OOM emergency borrowing notification process	55
Figure 7-7 Node fault detection framework	56

Tables

Table 7-1 RAS features 49

1 Introduction

1.1 Purpose

This document describes the architecture, functionality, and external interfaces of UnifiedBus (UB) Operating System (OS) Component. It is intended for application, driver, and OS developers, as well as anyone interested in the software architecture of UB OS Component.

1.2 Scope

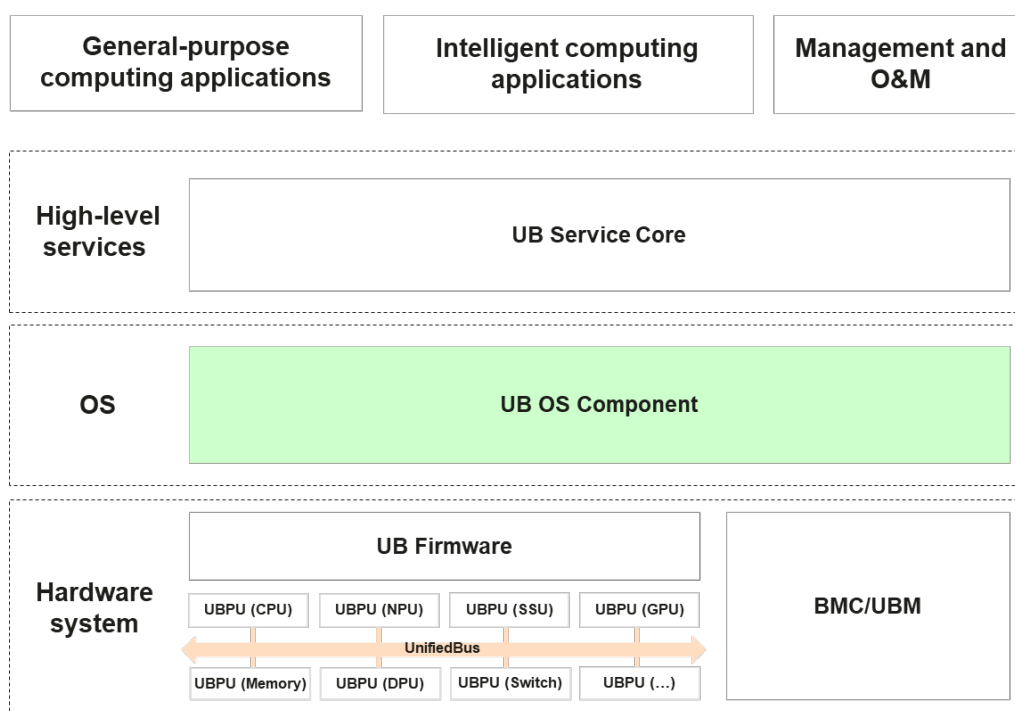


Figure 1-1 UB system diagram

The preceding figure shows the UB system. UB OS Component extends the original OS framework of memory management, communication, device management, and virtualization to better support UB. It implements unified abstraction and decoupling of heterogeneous hardware, provides a unified memory address space, supports global resource scheduling, dynamically combines and expands computing resources, and accelerates communication between devices, thereby unleashing the potential of UB hardware.

This document focuses on the architecture, functionality, and interfaces of UB OS Component.

- Architecture: software architecture and each layer of UB OS Component
- Functionality: functions at each layer and their relationships with existing OS components
- Interfaces: interface design and how to use these interfaces in typical scenarios

Note: For details about other components of the UB system reference architecture, download the corresponding documents from <https://www.unifiedbus.com/en>.

1.3 References

- UnifiedBus™ (UB) Base Specification 2.0
- UnifiedBus™ (UB) Firmware Specification 2.0

2 Architecture

The OS runs on the CPUs of compute nodes and provides a runtime environment for applications. The following figure shows the software architecture of UB OS Component.

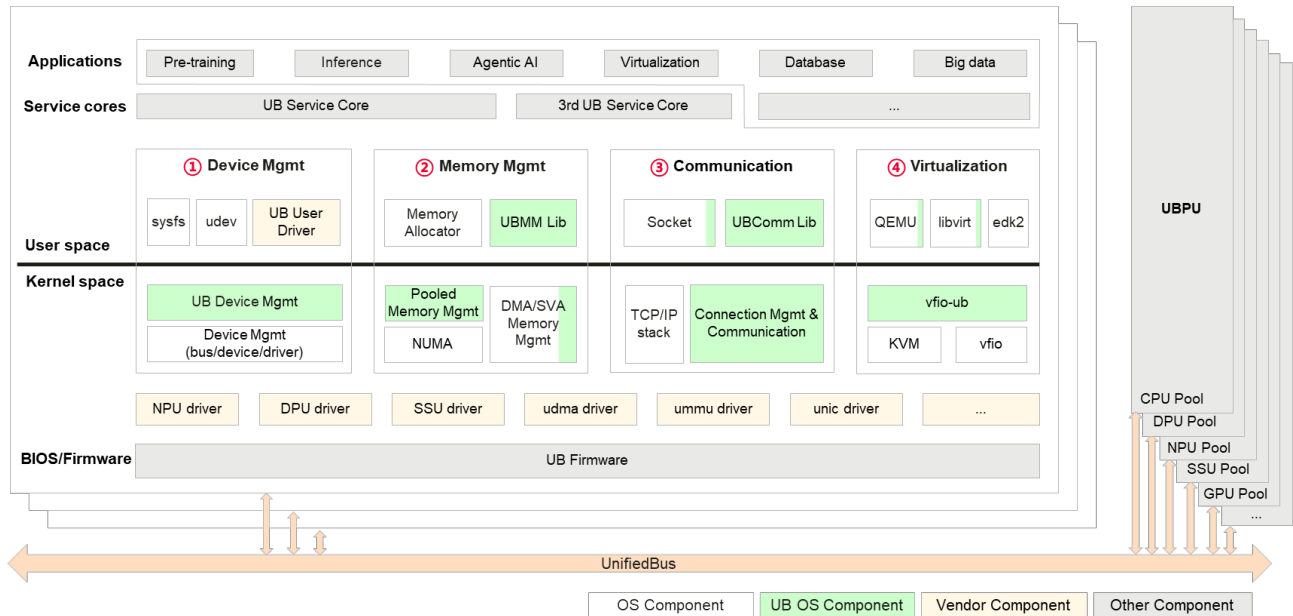


Figure 2-1 Software architecture of UB OS Component

UB OS Component enhances the original OS framework of memory management, communication, device management, and virtualization to provide improved support for UB. The feature enhancements include:

- ① **Device Mgmt:** It provides UB device management capabilities, allowing hot plugging and configuration of UB devices on compute nodes. It consists of the following modules:
 - **UB Device Mgmt:** Based on Linux's device management model (bus/device/driver), this module provides extended management for UB devices. It manages UB drivers, UB firmware interactions, and UB device hot plugging. In addition, it provides device driver interfaces for developing UB device drivers.
 - **sysfs:** UB Device Mgmt generates UB device, driver, and bus information in sysfs. Users can view and use the information through sysfs.
 - **udev:** UB Device Mgmt generates uevents. Users can use the udev tool to obtain the hot plug status of UB devices and manage these devices accordingly.
 - **UB User Driver:** It is the user-space driver for UB devices. It exposes UB devices to user space, enabling applications and virtualization software to use these devices.

For details, see 3 Device Mgmt.

- ② **Memory Mgmt:** It enables memory-semantic access for memory borrowing and sharing across compute nodes and devices within UB Domain. It consists of the following modules:
- **DMA/SVA Memory Mgmt:** Based on Linux's direct memory access (DMA) and shared virtual addressing (SVA) features, this module manages UB Memory Management Unit (UMMU). Drivers and other components can register host memory with UB devices using this module.
 - **Pooled Memory Mgmt:** Its functions include exporting local memory, importing remote memory, and maintaining shared memory consistency. Based on Linux's NUMA memory management framework, Pooled Memory Mgmt can bind remote memory to NUMA nodes. When it instead binds remote memory to memory devices (character devices created under **/dev**), the remote memory can be accessed through mmap calls.
 - **UBMM Lib:** It encapsulates the interfaces provided by Pooled Memory Mgmt in user space. UB Pooled Resource Manager (UBPRM, see Note 1) can call these interfaces to manage memory on both the Home and User sides.
 - **Memory Allocator:** It is compatible with common memory allocators including glibc, jemalloc, tcmalloc, and libnuma. After UBPRM borrows memory from other compute nodes and binds it to NUMA nodes through UBMM Lib, applications can allocate or release the borrowed memory using standard memory management interfaces such as malloc, free, and numa_alloc_onnode provided by these common allocators, without requiring any application modifications.

For details, see 4 Memory Mgmt.

- ③ **Communication:** It implements communication and remote procedure calls (RPCs) across compute nodes and UB devices. It consists of the following modules:
- **Connection Mgmt & Communication:** They manage asynchronous communication connections and provide kernel-space asynchronous communication interfaces for other modules.
 - **UBComm Lib:** It provides interfaces for enabling asynchronous communication and Unified Remote Procedure Call (URPC) within UB. Applications can use these interfaces to perform inter-node communication and function invocation within a SuperPoD.
 - **Socket:** It is compatible with glibc and other libraries that provide socket interfaces. Applications on different nodes can communicate with each other through socket interfaces without modification or with minimal modification (see Note 2).

For details, see 5 Communication.

- ④ **Virtualization:** It enables UB devices to be passed through to VMs. It consists of the following modules:
- **QEMU:** It extends QEMU functions to virtualize UB Controller, UB devices, and UMMU.
 - **libvirt:** It extends libvirt functions to support UB device creation and deletion.
 - **vfio-ub:** It supports UB device virtualization based on Linux's Virtual Function I/O (VFIO) framework.

For details, see 6 Virtualization.

Note 1: UBPRM manages pooled resources within a SuperPoD. It schedules resources in response to allocation requests initiated by a cloud management platform such as Kubernetes or by a configuration administrator—for example, allocating certain resources from node A to node B. It also supports automatic scheduling. For example, UBPRM can select a memory lender node in a SuperPoD based on a preconfigured scheduling policy and invokes UB Fabric Manager (UBFM) to allocate the designated resources to the target node. UBPRM is implemented by users. UBS Engine provided by UB Service Core is one such implementation.

Note 2: Applications can directly use UB communication modules without binary modification, or may require preloading or minimal code adaptation.

3 Device Mgmt



3.1 Overview

UB device management involves UBus Driver, vfio-ub, and ubutils, which together provide a set of UB device service management interfaces. These services include device discovery, device registration, and interrupt enabling for UB device drivers; user-space passthrough to UB devices; and querying UB device information and configurations. Various UB devices can be registered with UB to perform their functions. In the OS, UB device management functions, such as device discovery, registration, and driver loading, can be used in both single-server and cluster deployments.

- Single-server scenarios: Each server operates independently and exclusively manages its local UB devices. UBus Driver discovers and enables these devices, while vfio-ub enables user-space passthrough for direct device access.
- Cluster scenarios: In a SuperPoD, UBus Driver discovers and enables UB Entities allocated to compute nodes, supports the addition and removal of pooled devices, and works with vfio-ub to enable user-space passthrough for direct device access.

Note: UBPRM and UBFM work together to allocate UB Entities to compute nodes.

3.2 Functional Architecture

3.2.1 Device Management Architecture

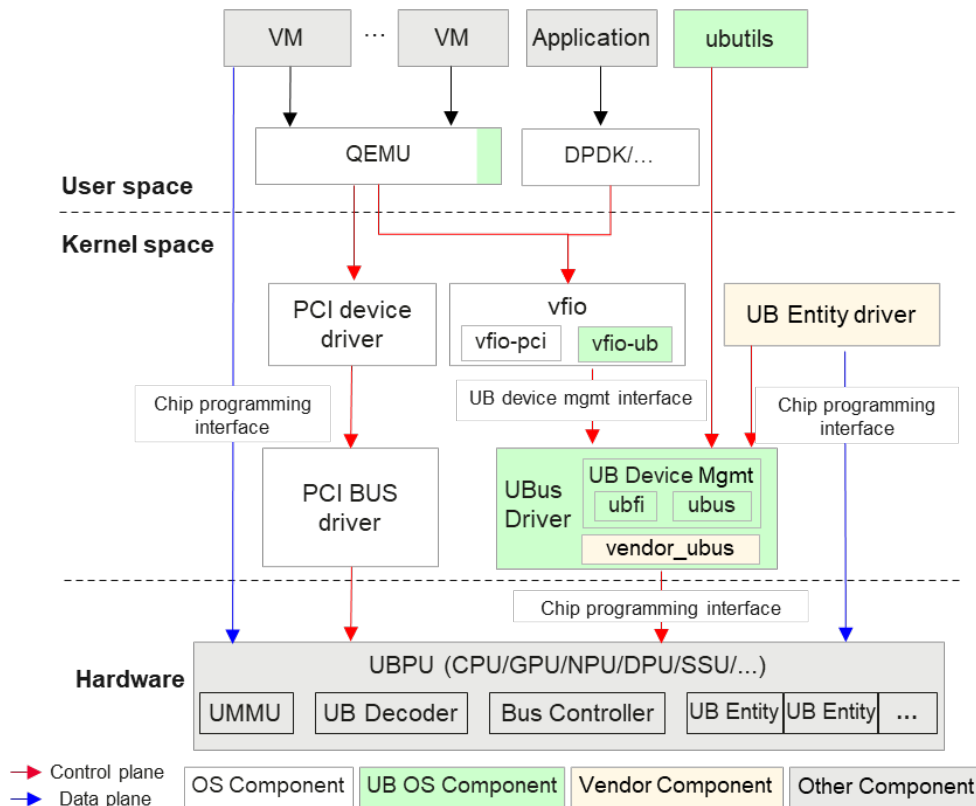


Figure 3-1 UB device management architecture

- **UBus Driver:** It consists of **ubfi**, **ubus**, and **vendor_ubus** modules. **ubfi** parses the **UBRT** table reported by the BIOS, registers platform devices, and generates the required Bus Controller. **ubus** provides basic device services based on Bus Controller, including UB device enumeration, UB device creation, error handling, hot plugging, and interrupt handling. **vendor_ubus** is a vendor-specific implementation responsible for interacting with the vendor's hardware.
- **vfi-ub:** It enables user-space passthrough for UB devices.
- **ubutils:** It provides UB device management tools to support UB topology query, configuration space query, and device configuration.

The major device management functions are as follows:

- **UB device information parsing:** **ubfi** parses the **UBRT** table reported by the BIOS. This table contains SoC information related to UB components, including register addresses, associations of UB devices with the interrupt controller and with the UMMU, and system resources that can be allocated by UB. After obtaining information from the **UBRT** table, the OS parses the table entries, creates and registers device nodes such as Bus Controller and UMMU, scans for the Bus Controller ports, completes UB Entity enumeration, and establishes the topology relationships.

- Device enumeration: UBus Driver enumerates UB devices within a single server without repetition or omission. Device enumeration requires using Neighbor Discovery Protocol, enumeration management messages, and configuration access messages. This process includes topology scanning, network address allocation, routing table setup, Entity Identifier (EID) assignment, and initialization of discovered UB Entities for registration on UB.
- Device resource space management: The host obtains device resource space information and allocates its address spaces to devices. The host driver accesses the device resource space using the host-side address. The CPUs allocate an address window to each UB Entity. UBus Driver manages the address resource, allocates an address space to each device when UB Entities are enabled, and releases the address space after the device is removed. UB devices support hot plugging and UB Entities are created on demand. Therefore, dynamic resource allocation and release shall be supported.
- Device configuration space access: UB Entities support configuration space registers, which store capability description and configuration interfaces for device management. According to the UB protocol, configuration access messages are used to read from and write to configuration space registers. Bus Controller acts as the initiator of configuration access messages, while the device serves as the responder. Bus Controller also has its own configuration space register and functions as both the initiator and the responder. UBus Driver accesses configuration space registers using hardware and software interfaces (Message Queue) provided by Bus Controller.
- Bus device driver: The UB device driver framework leverages the Linux kernel's driver model to enable unified management of UB devices and complies with the bus/device/driver architecture of that model. It provides a unified abstraction of UB Entities and standardized programming interfaces that allow vendor drivers to manage devices. It also manages UB services and offers common service registration and deregistration interfaces for UB components, such as Reliability, Availability, and Serviceability (RAS) and hot plugging.
- Device interrupts: UB interrupts adopt a message-based mechanism to establish interrupt paths between UB devices and the system. They handle access to the interrupt controller and manage UB device interrupt requests and releases.
- Port management: UBus Driver allows querying and configuring information related to port link establishment and neighboring devices. These operations can be initiated by either the driver software or the user.
- Error detection and reporting: UBus Driver supports the error detection, correction, and reporting mechanisms defined in the *UnifiedBus™ (UB) Base Specification 2.0*. These mechanisms enable efficient error handling and enhance overall system robustness and reliability.
- UB device information query and configuration: UB provides the lsub and setub tools to view the UB network topology and device configuration spaces, as well as to modify a specified device configuration space. It exposes UB subsystem information to user space through sysfs and grants the write permission on certain modules, allowing the kernel to be triggered to execute related tasks.

3.2.2 Device Driver Model

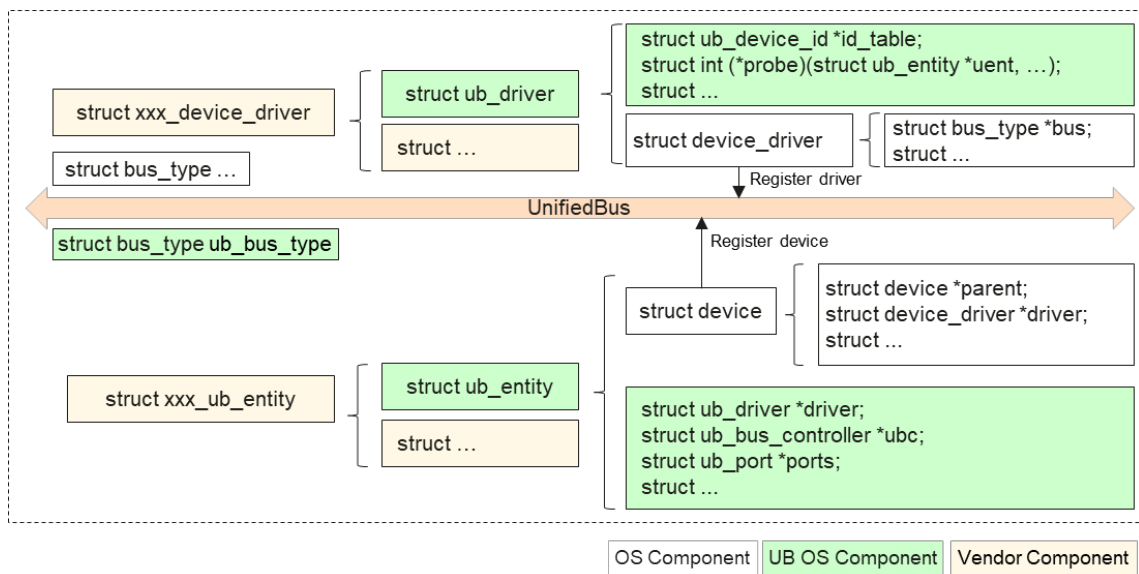


Figure 3-2 UB device driver model

The UB device driver model adopts the Linux kernel driver framework to enable unified management of UB devices. Following the bus/device/driver design, it provides a standardized abstraction for UB devices and unified programming interfaces for vendor drivers. The UB device management model defines the following data structures:

- **ub_bus_type:** It implements the set of operations defined by **bus_type** in the driver model. UB is registered with the OS during OS startup.
- **struct ub_entity** (including **struct device**): It describes UB Entities. Each UB Entity corresponds to an instance of `ub_entity`.
- **struct ub_driver** (including **struct device_driver**): It describes UB Entity drivers.

After a UB Entity is inserted, the system creates a `ub_entity` and matches it with a `ub_driver`. It presents the following device tree to users:

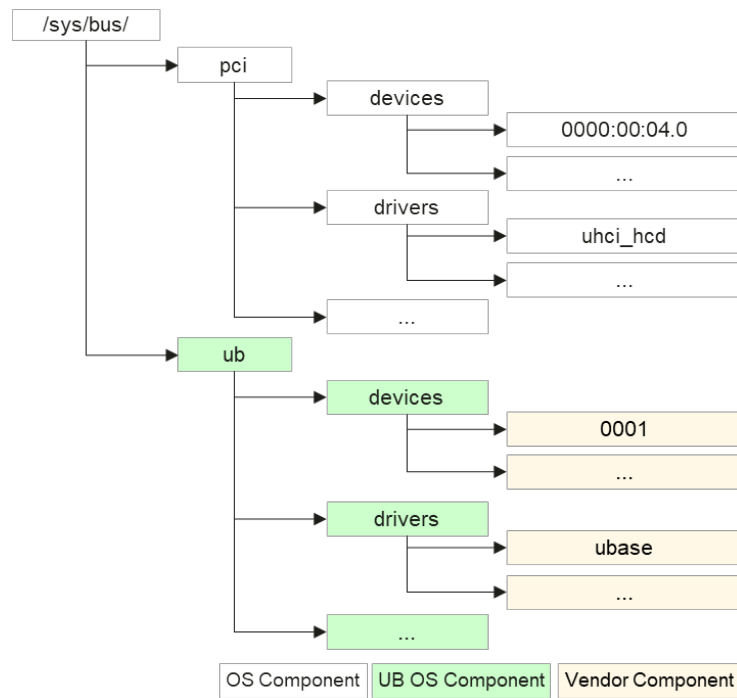


Figure 3-3 UB device tree

3.2.3 Device Service Model

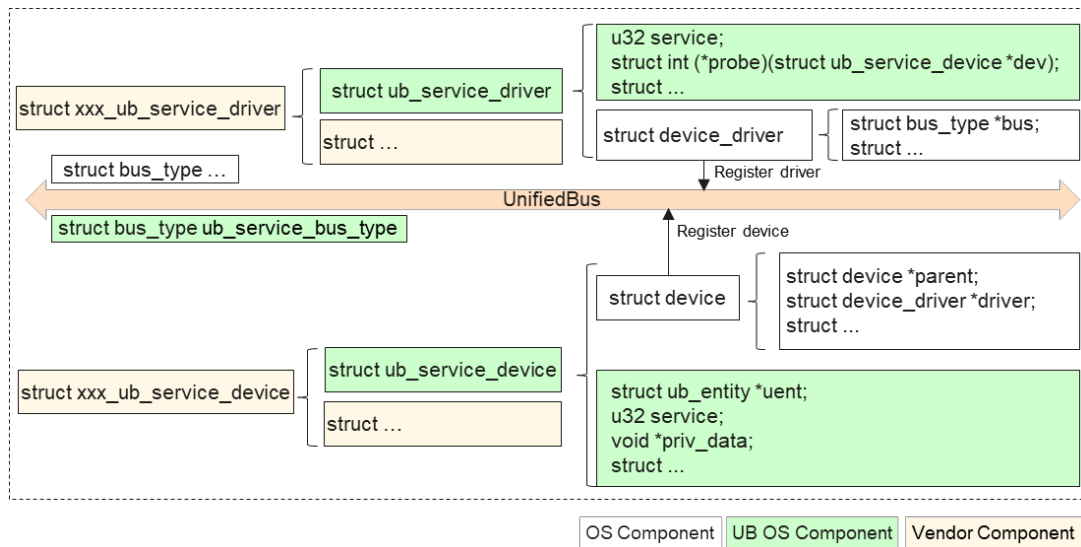


Figure 3-4 UB device service model

The UB device service management function is used to register common UB device services, such as RAS and hot plugging, with the drivers of UB components, including UB Controller and UB Switch. Device services are registered with `ub_service` based on the component capability in the UB Entity drivers. They are matched with UB device service drivers and incorporated into the specific processing routines of the device services. The UB device service defines the following data structures:

- **ub_service_bus_type**: It implements the set of operations defined by **bus_type**. UB is registered with the OS during OS startup.

- **struct ub_service_device** (including **struct device**): It describes the UB device service and creates `ub_service_device` instances for RAS and hot plugging.
- **struct ub_service_driver** (including **struct device_driver**): It describes UB device service drivers.

The device service management framework provides UB component drivers with interfaces for registering and deregistering UB device service drivers. It offers only service registration and deregistration functions. Consequently, **ub_service_bus_type** defines only the match operation interface, which uses an ID-based mechanism to match services according to their service type. It presents the following device tree to users:

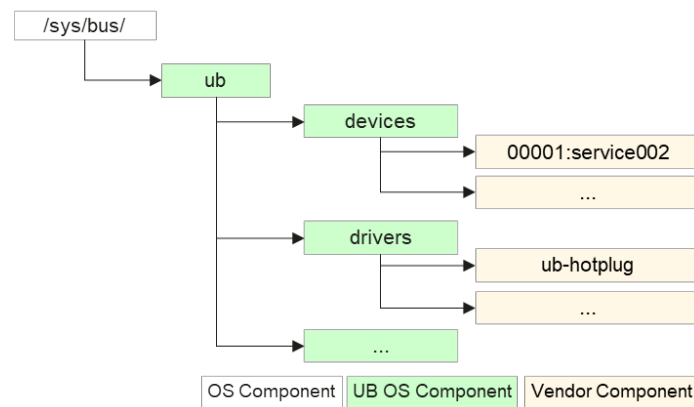


Figure 3-5 UB service device tree

3.3 Usage Process

3.3.1 Device Drivers

The UB driver is registered with the system by calling `ub_register_driver`, which is generally implemented in the **init** function of the driver module. Once UB detects a UB device and matches it with a driver, the driver generally performs the following initialization process (generally implemented in the **probe** function of the driver):

1. Call `ub_set_user_info` to set host information, such as an EID.
2. Call `ub_entity_enable` to enable UB Entities.
3. Call `ub_iomap` to map a resource space.
4. Access the device configuration space to initialize the device.
5. Call `request_irq` to register the interrupt request (IRQ) handler.
6. Register the UB Entities with other subsystems (such as NVMe or netdev).

When the device is no longer used, the driver needs to remove the device (generally implemented in the **remove** function of the driver):

1. Deregister the UB Entities from other subsystems (such as NVMe or netdev).
2. Disable the IRQs generated by the device and call `free_irq` to release them.

3. Call `ub_iounmap` to release the resource space.
4. Call `ub_entity_enable` to disable the device.
5. Call `ub_unset_user_info` to clear the host information.

When the entire driver module needs to be removed, call `ub_unregister_driver` to deregister the device driver (generally implemented in the **exit** function).

Note: Not all drivers require the preceding steps. Driver developers can adjust the initialization and removal steps based on the device functions and design.

3.3.2 Device Service Driver

The process of developing and using a device service driver is as follows:

1. The UB service driver registers with the system through `ub_service_driver_register`.
2. After the UB component device is detected, `ub_service_bus_type` matches the service ID with a registered driver. Once matched, it calls the **probe** function of the corresponding driver to initialize the service.
3. `ub_service_bus_type` calls the **remove** function of the corresponding driver to remove the UB component device.
4. To remove the entire driver module, `ub_service_driver_unregister` is called to unregister the driver.

3.4 Maintenance and Debugging

- UBus Driver
 - (1) Administrators use `ubutils` tools to query and configure UB Entities and topologies. The `lsub` tool is used to view device topologies and device configuration spaces, whereas `setub` is used to configure device spaces.
 - (2) Administrators use the `sysfs` interface to query information, including UB Entity information such as EIDs, UB partition identifiers (UPIs), and globally unique identifiers (GUIDs).
- UB Entity driver
 - (1) Device and service drivers generate logs during initialization and removal, which can be viewed by running the **`dmesg`** command.
 - (2) In user space, applications can inspect the device tree, query device attributes, and control devices via `/sys/bus/ub` and `/sys/bus/ub_service`.

4 Memory Mgmt

4.1 Overview

This chapter describes the software architecture and interfaces that enable memory access within the UB interconnect topology.

In general, existing bus architectures follow a host-device model, where the host (CPU) manages each device and three independent memory access paths exist: host-to-device, device-to-host, and device-to-device. UB streamlines this model. It categorizes UB nodes as Users and Homes in the memory access model. Users can access the memory resources of Homes either synchronously (via load/store) or asynchronously (via DMA).

4.2 Functional Architecture

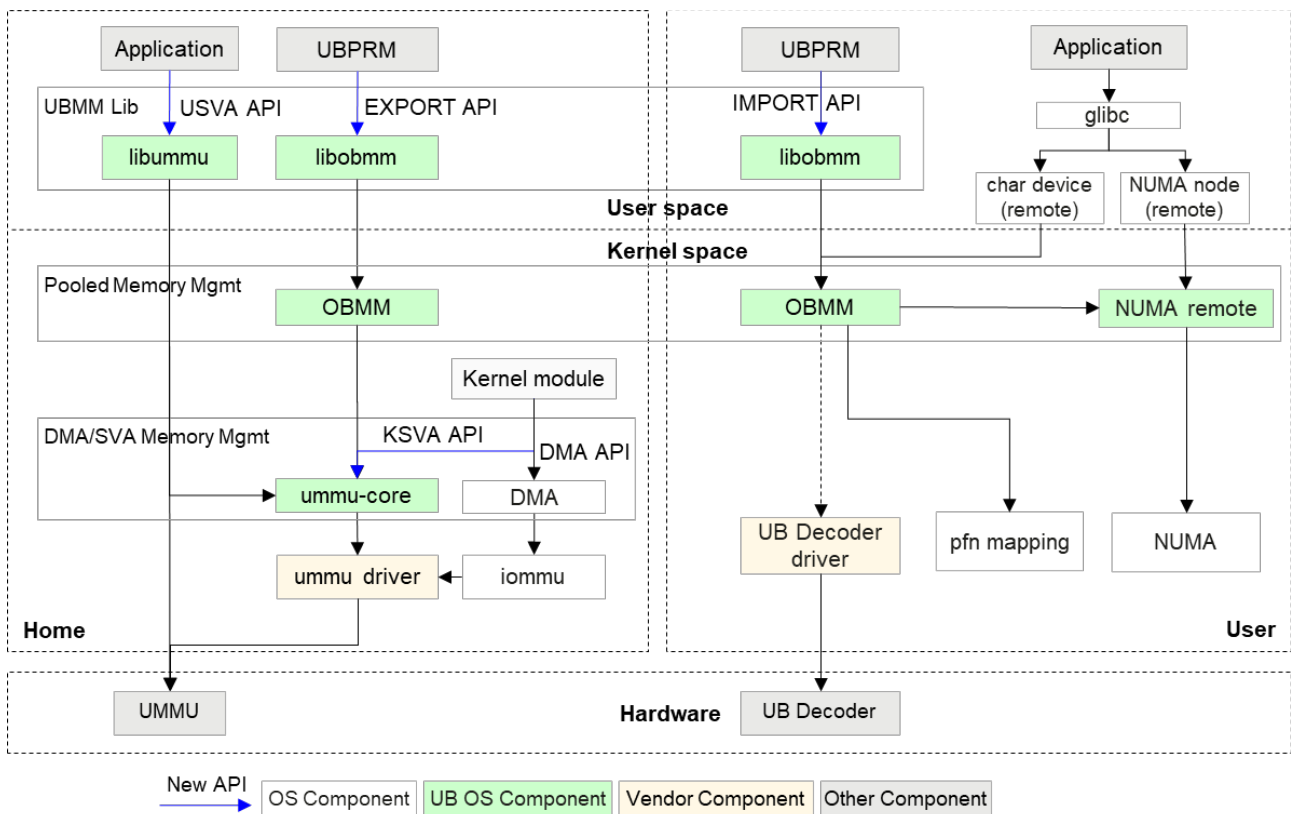


Figure 4-1 UB Memory Management (UBMM) software architecture

- **libummu:** It provides APIs for allocating and releasing token IDs in user space, and maintains the permission table defined in the *UnifiedBus™ (UB) Base Specification 2.0*.
- **ummu-core:** It provides basic token ID management, as well as configuration and management of advanced capabilities defined in the *UnifiedBus™ (UB) Base Specification 2.0*.

- **ummu driver:** It interacts with the UMMU hardware, implements the underlying capabilities defined by ummu-core, and ensures memory paths on the Home side.
- **libobmm/OBMM:** It provides memory borrowing interfaces (EXPORT and IMPORT), as well as memory consistency maintenance interfaces (used only for memory sharing).
- **UB Decoder driver:** It configures the UB Decoder hardware to ensure proper operation of load/store paths on the User side.

Note: ummu driver and UB Decoder driver are used to configure memory paths on the Home side and User side, respectively. In addition, links between Homes and Users can be established only after the correct route information is configured in UBFM.

4.2.1 Memory Access Target (Home)

As described in the *UnifiedBus™ (UB) Base Specification 2.0*, a memory access target needs to register local memory resources with UMMU and generate a memory access credential tuple {EID, TokenID, UBA}. The OS provides four types of interfaces to perform these operations:

- **User Shared Virtual Address (USVA):** It exports the specified VA segments of a user-space process to other devices or nodes. USVA uses a permission table to restrict device access to specified VA segments in user space.
- **Kernel Shared Virtual Address (KSVA):** It exports the specified VA segments in kernel space to other devices or nodes. KSVA uses a permission table to restrict device access to the allowed VA range in kernel space.
- **DMA:** It maps the specified physical memory as UB addresses in kernel space and exports them to other devices or nodes.
- **EXPORT:** It exports local memory resources of a specified size to other devices or nodes.

4.2.2 Memory Access Initiator (User)

A memory access initiator uses an access credential to initiate memory access through multiple methods.

- **Asynchronous access:** The User employs an access credential tuple provided by the Home to initiate asynchronous DMA operations through the DMA or Unified Remote Memory Access (URMA) engine.
- **Synchronous load/store access:** The User calls the IMPORT interface to configure the access credential tuple into UB Decoder. Afterward, all load/store access requests to a specified physical address are directed to the Home through UB.

The IMPORT interface takes the memory resources exported by the EXPORT interface as input parameters, and uses them in either of the following methods:

- After memory resources are bound to the NUMA nodes of the local OS, they are assigned to user-space processes through a standard memory allocation interface (for example, malloc/free in glibc or jemalloc).

- After memory resources are bound to character devices, user-space processes can access them by mapping them into their address spaces using mmap.

4.3 Usage Process

4.3.1 EXPORT/IMPORT

The EXPORT and IMPORT interfaces are used to export and import memory during memory borrowing. In a SuperPoD, the memory resources of nodes with ample memory can be exported, and then imported to nodes that have insufficient memory or need additional memory for higher performance.

UBPRM makes a memory borrowing decision. To do so, it needs to obtain the EIDs on the Home side, the EIDs on the User side, and the size of the memory to be borrowed. The following figure illustrates the memory borrowing process:

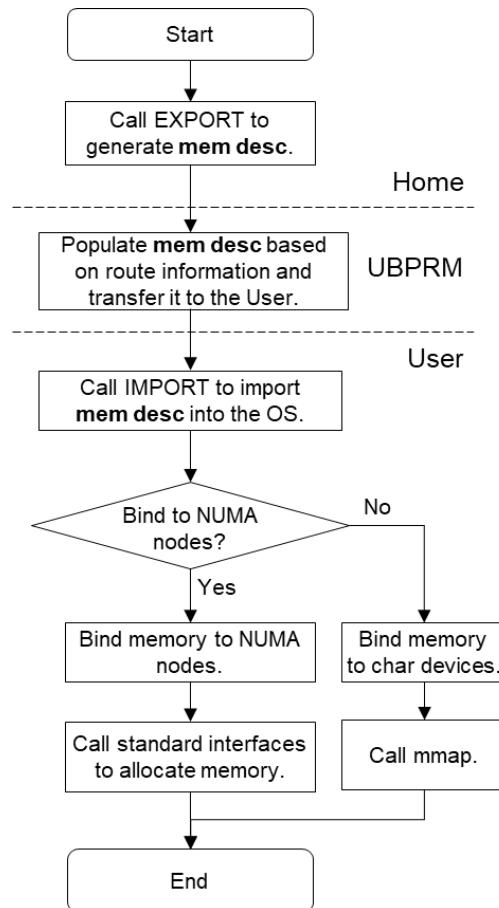


Figure 4-2 Memory borrowing process

1. Home: It calls the obmm_export API, specifies the EID and memory size, and generates the UBA and token ID information in **mem desc**. The tuple {dest EID, UBA, TokenID} describes the memory lending information on the Home side.
2. UBPRM: It populates {src EID, scena, dcna} and private information implemented by different vendors in **mem desc**.

3. User: It calls the obmm_import interface to import **mem desc** into the OS.
4. User: To bind memory resources to NUMA nodes, it calls the standard memory APIs (such as malloc, mbind, numa_alloc_onnode, and move_pages) to allocate and use memory.
5. User: To bind memory resources to character devices, it calls the mmap interface to map them as address spaces.

4.3.2 OWNERSHIP

When multiple nodes need to share data for reading and writing, one node may export a memory segment, while other nodes import and use that segment, thereby enabling memory sharing within a SuperPoD. If the hardware between nodes does not maintain cache coherence, there is a risk of data inconsistency when multiple applications read or write data simultaneously. In this case, a hardware-software collaboration mechanism is required to ensure data consistency.

OWNERSHIP is used to manage the holding status of a memory segment. Each node has an independent local OWNERSHIP state for each memory segment. OWNERSHIP has three states:

- **Invalid:** This node cannot read from or write to the memory.
- **Write:** This local node can read from and write to the memory.
- **Read:** This node can read from the memory.

The application needs to proactively transition the OWNERSHIP state based on memory read/write requirements to ensure that software operations on memory comply with the definitions of these OWNERSHIP states. OWNERSHIP state transitions involve the following operations:

- **Clean:** Dirty cachelines are written back to memory and retained.
- **Invalidate:** Specified cachelines are invalidated, and subsequent CPU accesses read them directly from memory.

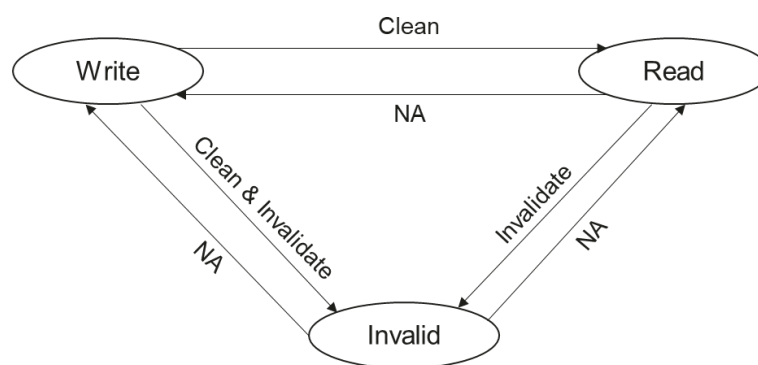


Figure 4-3 OWNERSHIP state transitions

OWNERSHIP state transition operations:

- **Write -> Invalid:** The Clean and Invalidate operations ensure that the modified data is written back to memory and the corresponding cachelines are invalidated.

- **Write -> Read:** The Clean operation ensures that the modified data is written back to memory and the corresponding cachelines are retained to accelerate subsequent read operations.
- **Read -> Invalid:** The Invalidate operation ensures that the corresponding cachelines become invalid while memory data is not affected.
- Others ("NA" state transitions marked in Figure 4-3): No action is required.

OBMM provides the `obmm_set_ownership` interface to implement the OWNERSHIP mechanism. When using shared memory, ensure that only one application performs write operations at a time. In addition, call the `obmm_set_ownership` interface to maintain data consistency before and after reading or writing a memory segment.

The process for memory sharing on the Home side is the same as that for memory borrowing. On the User side, the `obmm_import` interface is called to bind memory resources to a character device. Then applications obtain the VA through `mmap` and access the remote memory directly.

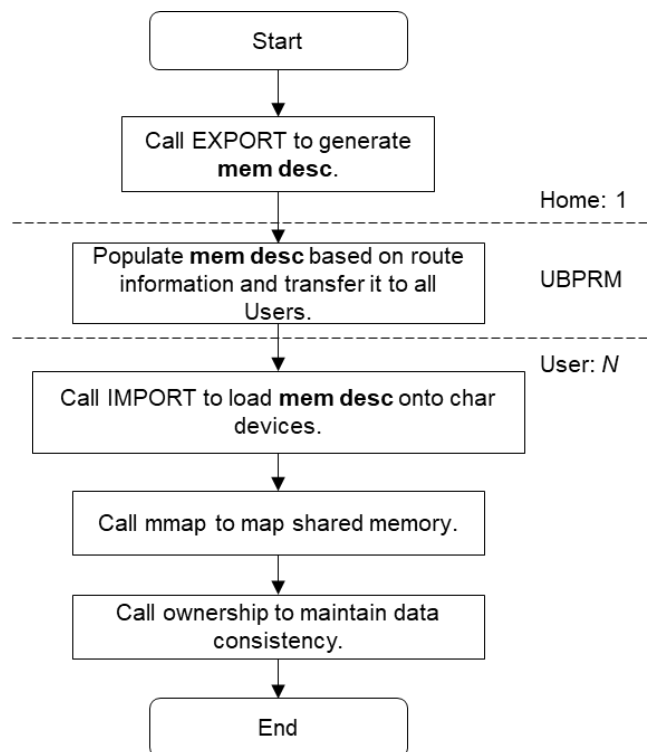


Figure 4-4 Memory sharing process

1. Home: It calls the `obmm_export` API, specifies the EID and memory size, and generates the UBA and token ID information in **mem desc**. The tuple {dest EID, UBA, TokenID} describes the memory lending information on the Home side.
2. UBPRM: It populates **mem desc** based on the topology and route information of the Home and each User.
3. User: It calls the `obmm_import` interface to load **mem desc** onto character devices.
4. User: It calls the `open` and `mmap` interfaces to obtain the VA of the shared memory in user space.

5. User: It calls the `obmm_set_ownership` interface (only in cacheable mode) to change the OWNERSHIP state before or after a read or write operation to maintain data consistency.

4.3.3 DMA/KSVA

Generally a device driver uses the DMA interface or SVA to apply for a Home-side resource, and then allocates the resource to a device using a device configuration space. The device driver in UB follows the same approach.

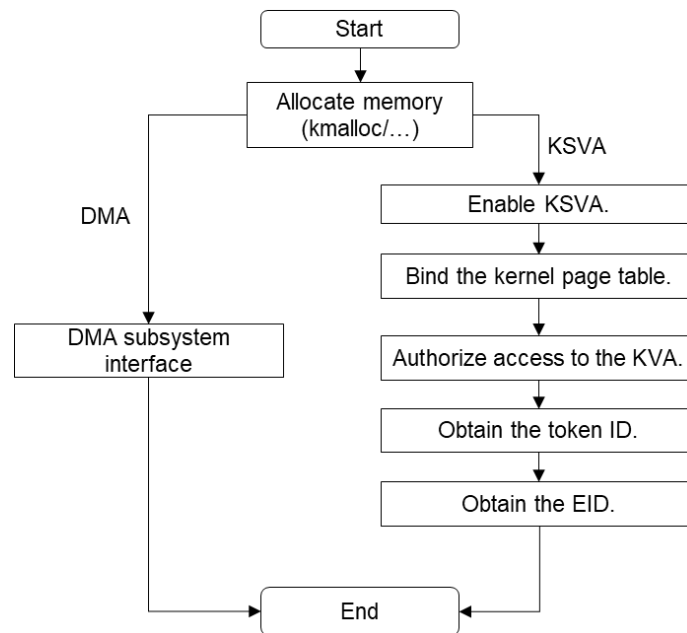


Figure 4-5 Allocating Home-side memory to a device

The two processes illustrated in the preceding figure are as follows:

- Using Home-side memory through DMA
 - (1) Request or map local memory resources through the DMA interface of Linux to obtain an I/O virtual address.
 - (2) Configure the I/O virtual address as a UB address into the device register.
- Using Home-side memory through KSVA.
 - (1) Enable the KSVA feature.
 - (2) Bind the kernel page table using the KSVA bind interface.
 - (3) Grant the User access to designated memory resources through the authorization interface.
 - (4) Configure {EID, TokenID, UBA, TokenValue} for the device.

4.3.4 USVA

In user space, a UB memory descriptor (EID, TokenID, UBA, where UBA equals VA) can be generated for memory resources in the process address space. This enables the implementation of a user-space device driver.

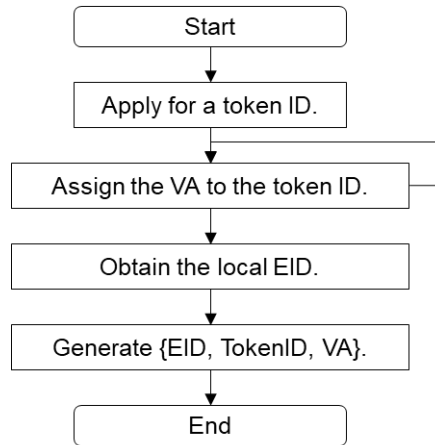


Figure 4-6 Creating a UB memory descriptor in user space

1. Call the `ummu_allocate_tid` interface to apply for a token ID.
2. Call the `ummu_grant` interface to assign the VA of the process to the token ID. This step can be performed multiple times.
3. Obtain the EID of the local host node or device node.
4. Generate the complete {EID, TokenID, UBA} information for other nodes or devices.

4.4 Maintenance and Debugging

UB Memory Mgmt builds upon established Linux maintenance and debugging mechanisms, such as `ftrace` and `kprobe`. It can also use `sysfs` to facilitate maintenance and debugging.

- Under `/sys/devices/platform/ummu_tid_root`, each created token ID has a dedicated directory that contains detailed information about the token ID.
- Under `/sys/devices/obmm`, each created memory ID has a dedicated directory that provides detailed information about the memory ID.

4.5 UBTurbo

4.5.1 Overview

UBTurbo provides performance acceleration capabilities for the OS.

- Remote memory access has higher latency than local memory access. Therefore, careful planning of remote memory usage is essential to prevent significant performance fluctuations. UBTurbo provides the SMAP component as a reference for migrating data between remote and local memory. The core idea is to migrate page data based on the local

spatiotemporal characteristics of application access patterns, combined with the hardware-assisted hot and cold data identification capabilities provided by the chip (the chip shall support hot and cold data identification for remote memory). Hot data is stored in local memory while cold data is placed in remote memory, thereby delivering end-to-end performance acceleration for applications.

- In a hierarchical memory scenario, the access latency of remote memory is higher than that of local memory. Therefore, the location and proportion of remote memory need to be carefully planned. UBTurbo provides a remote-to-local memory ratio adjustment feature. It can dynamically modify this ratio based on memory latency sensitivity and the distribution of hot and cold data.

4.5.2 Module Architecture

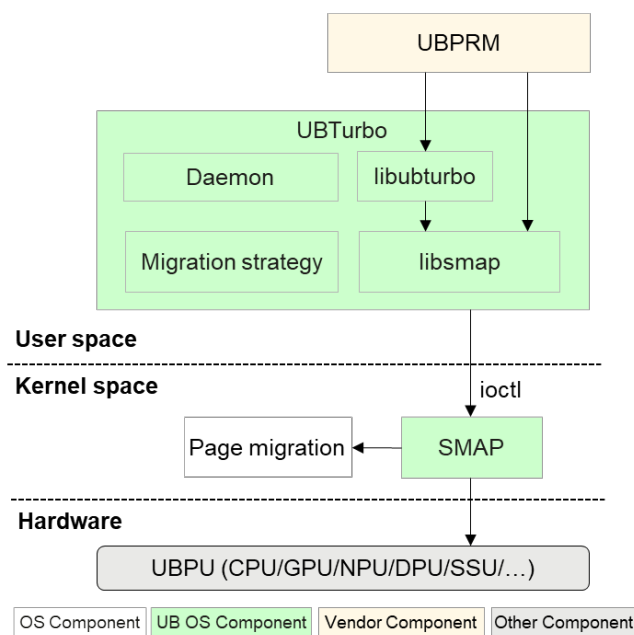


Figure 4-7 UBTurbo module architecture

- **libubturbo:** This user-space library provides APIs for core functions, such as adjusting the remote-to-local memory ratio and implementing SMAP.
- **libsmmap:** It migrates data based on the collected hot and cold data statistics to improve performance in hierarchical memory scenarios.
- **Daemon:** It is the UBTurbo process service.
- **MigrationStrategy:** It provides resource migration scheduling policies.
- **SMAP:** It provides the kernel execution capability for page migration and collects hot and cold data information provided by the chip.

4.5.3 Usage Process

4.5.3.1 SMAP

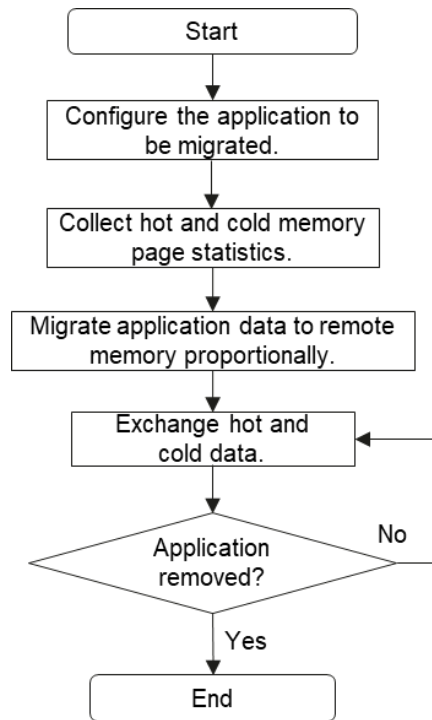


Figure 4-8 Hot and cold data identification and migration

1. Call the SmapInit interface to initialize the SMAP component and start collecting hot and cold memory page statistics.
2. Call the SetSmapRemoteNumaInfo and SmapMigrateOut interfaces to migrate application data to remote memory proportionally.
3. SMAP provides a reference policy to periodically exchange hot and cold data for application processes registered in the migration allowlist, achieving optimal memory access performance under specific ratios.
4. When an application needs to migrate data from remote memory, call the SetSmapRemoteNumaInfo and SmapMigrateBack interfaces to migrate the application data back to local memory.

4.5.3.2 Adjusting the Remote-to-Local Memory Ratio

After remote memory is allocated, libubturbo controls hierarchical memory acceleration capabilities of UBTurbo. It determines the target location and size of the migrated process data in remote memory based on memory latency sensitivity and the distribution of hot and cold data. Additionally, it dynamically adjusts the remote-to-local memory ratio for the migrated process based on changes in hot and cold data, optimizing data access performance and resource utilization.

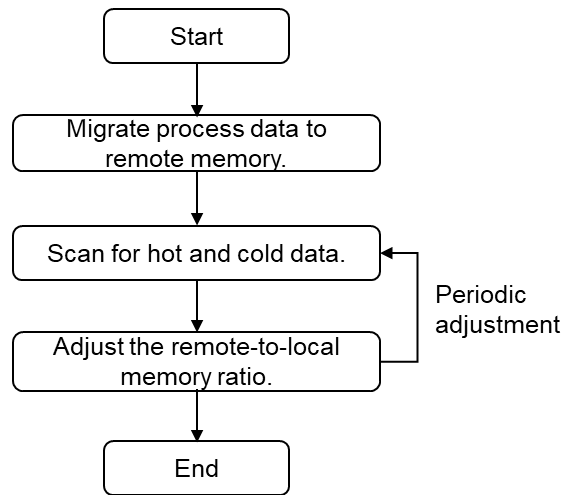


Figure 4-9 Adjusting the remote-to-local memory ratio

1. Call the OutSetWaterMark interface to initialize the scenario configuration.
2. Call the OutMemMigrate interface to submit the IDs of processes eligible to use remote memory, along with the maximum percentage of each process's data space that can be migrated out.
3. Call APIs such as SmapMigrateOut to migrate process data to the specified remote memory.
4. SMAP dynamically adjusts the remote-to-local memory ratio for processes based on the distribution of hot and cold data and the service performance requirements.

5 Communication

5.1 Overview

UnifiedBus Memory Development Kit (UMDK) is a distributed communication software library that provides high-performance communication interfaces for inter-card communication within data center networks, SuperPoD environments, and servers, fully utilizing UB hardware capabilities.

5.2 Functional Architecture

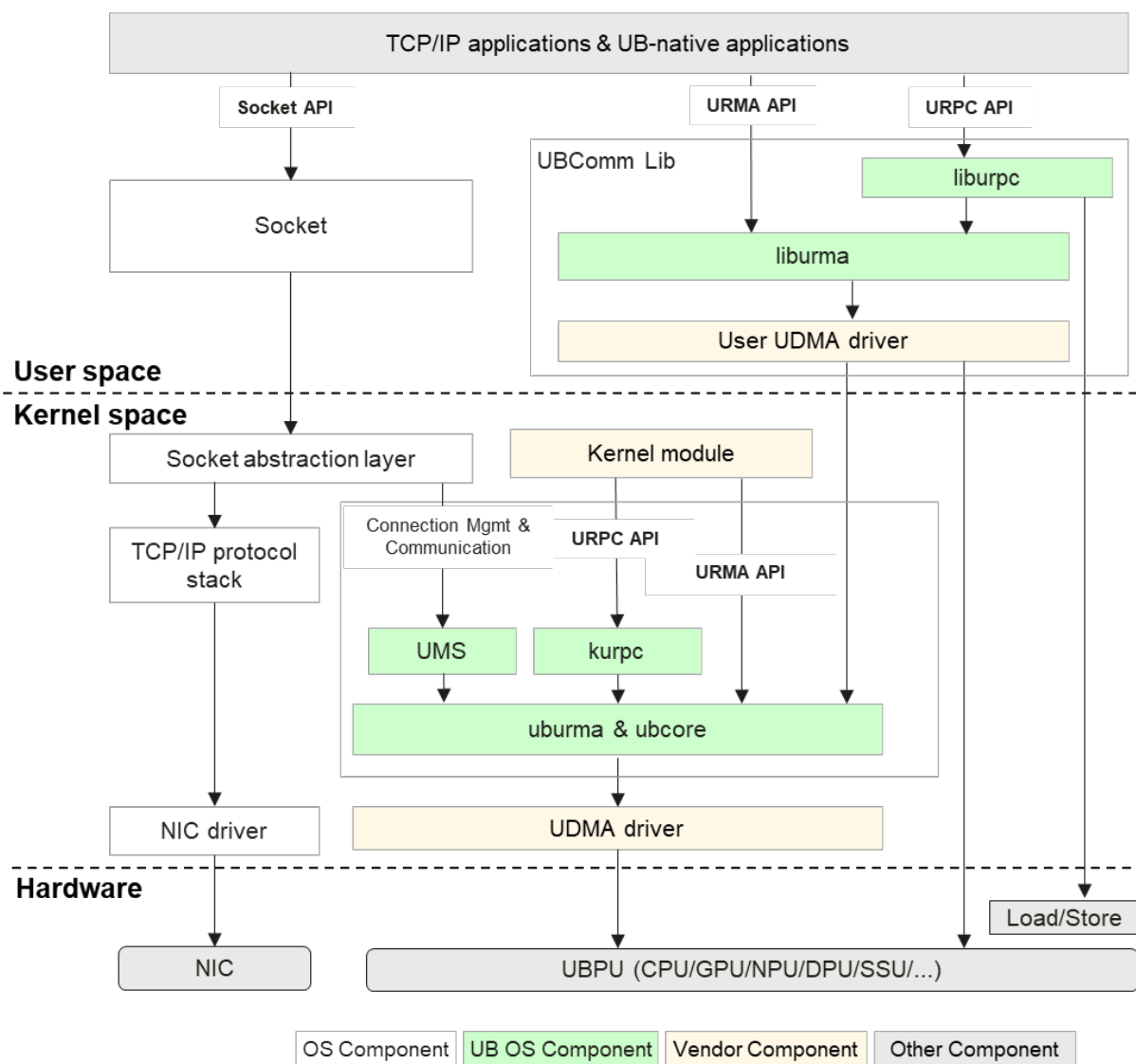


Figure 5-1 UMDK functional architecture

- **UDMA driver & User UDMA driver:** The device drivers provided by hardware vendors support kernel-space and user-space UMDK integration, respectively.

- **uburma & ubcore & liburma:** The uburma and ubcore modules run in kernel space, while liburma runs in user space. They provide remote memory access operations including one-sided, two-sided, and atomic operations, forming the foundation for inter-application communication. They expose northbound APIs for application communication, and southbound APIs for driver developers to access UMDK.
- **liburpc & kurpc:** They provide RPC semantic communication interfaces via URPC.
- **UMS:** It connects to the socket abstraction layer to implement socket compatibility.

5.3 URMA

5.3.1 Overview

In the servers, racks, and clusters of the UB peer-to-peer architecture, compute units such as CPUs, GPUs, and NPUs have their own advantages in processing different tasks. To fully leverage heterogeneous computing power and efficiently complete computing tasks, the following communication requirements arise:

- Direct communication among heterogeneous compute units for higher system performance: Peer-to-peer heterogeneous compute units scale out horizontally, where all components can directly communicate with each other and mutually invoke computing power, supporting fine-grained parallel processing or invocation.
- Large-scale high-performance computing and communication: The communication efficiency within and between nodes should be improved to support large-scale high-performance computing and communication.
- Flexible routing for enhanced network utilization and real-time performance: Multipath out-of-order transmission and efficient congestion control jointly unleash data center bandwidth capacity, improve network utilization, meet real-time service requirements, and enhance the overall efficiency of computing clusters.

URMA is designed to implement communication between two UB Entities, including one-sided DMA access and two-sided message sending and receiving. URMA has the following features:

- Peer-to-peer access: Any two heterogeneous computing devices can communicate directly via URMA, bypassing CPUs.
- Connectionless communications: URMA reuses the reliable services provided at the UB transport layer between applications, eliminating the need to establish end-to-end connections between applications and thus significantly cutting down on communication overhead.
- Relaxed transaction ordering and multipath transmission: URMA communication components allow applications to specify transaction ordering modes and support out-of-order transaction execution and completion, thereby avoiding transaction head-of-line blocking, enabling underlay multipath transmission, and significantly improving execution efficiency.

5.3.2 Module Architecture

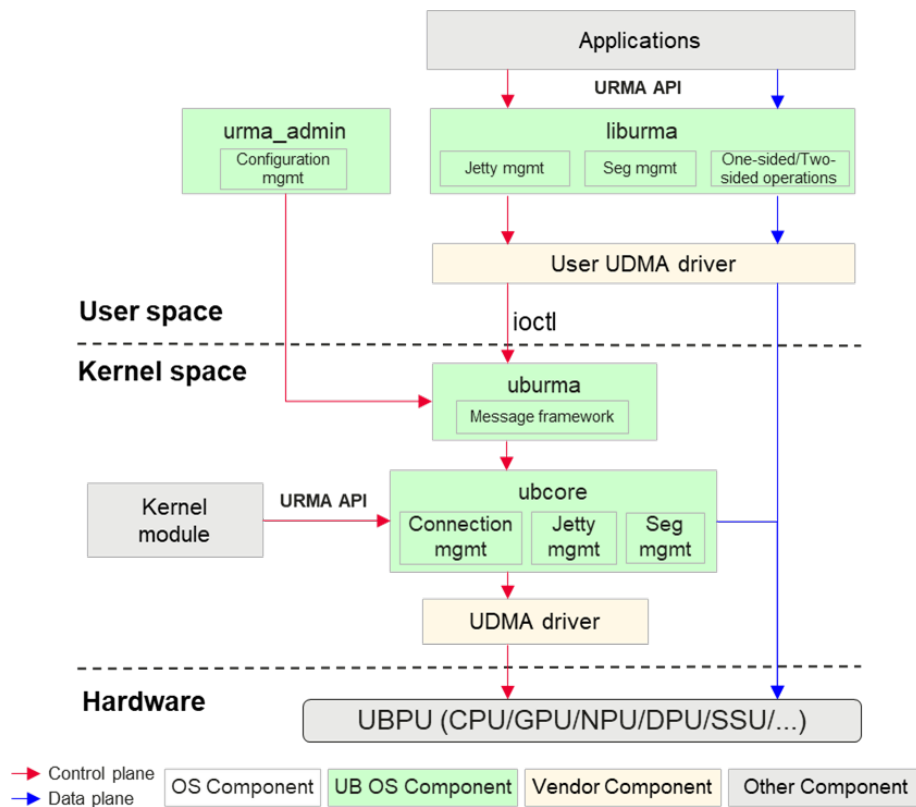


Figure 5-2 URMA module architecture

- **liburma:** The user-space communication library provides northbound APIs for core features such as Jetty management, segment management, and data-plane operations.
- **urma_admin:** The query and configuration tool manages URMA devices in the environment.
- **uburma:** It is a message-based communication framework between the user-space library and the kernel-space kernel layer.
- **ubcore:** It handles core functionalities such as connection setup, Jetty and segment resource allocation, and state management.
- **UDMA driver & User UDMA driver:** The device drivers provided by hardware vendors support kernel-space and user-space UMDK integration, respectively.

5.3.3 Usage Process

The overall process for using URMA is as follows:

1. Create the Jetty required for communication.
2. Create segments for reading and writing data.
3. Initiate communication via two-sided, one-sided, or atomic operations.
4. Read the corresponding completion records.

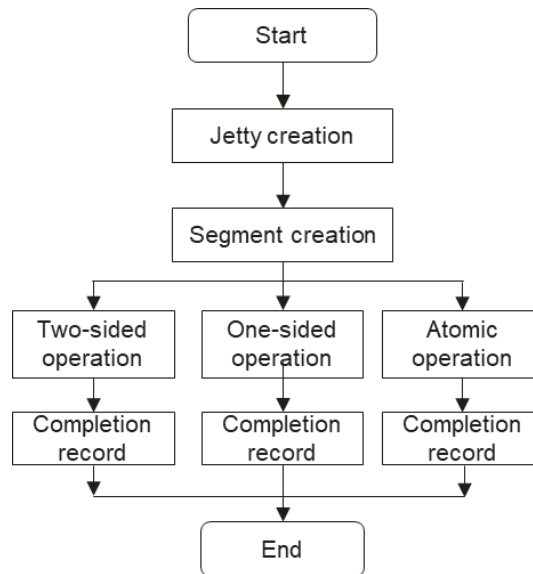


Figure 5-3 URMA usage process

5.3.3.1 Jetty Management

URMA performs resource management through Jetties. Jetties are used to receive and send messages, and deliver memory semantics commands. They can be classified into Jetty for Sending (JFS), Jetty for Receiving (JFR), Jetty for Completion (JFC), and Jetty for Completion event (JFCE) based on their functions. Necessary Jetty resources shall be created before performing subsequent operations, including read, write, send, and receive.

Jetty access is controlled through a token mechanism. On the management plane, the TokenValue is specified when creating the Jetty and exchanged with the sender via an application's secure channel. The sender configures the TokenValue on the hardware UB device when importing the Jetty. The UB device carries the TokenValue when sending messages on the data plane, and the receiver validates the token upon receiving access requests, granting access only if validation succeeds.

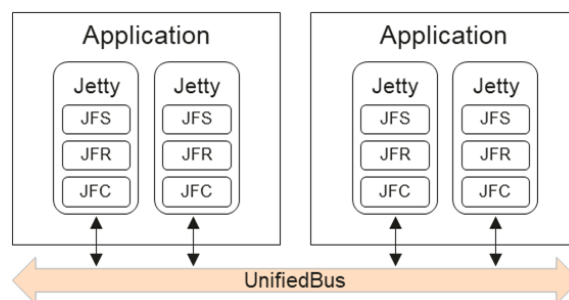


Figure 5-4 Jetty management

The overall process is as follows:

1. Call `urma_create_jetty` to create a local Jetty.
2. Call `urma_import_jetty` to import a remote Jetty.

5.3.3.2 Segment Management

Each segment is a contiguous VA space mapped to a physical memory region.

The token mechanism is used to protect memory security during one-sided operations (for details, see Section 5.3.3.4). When registering a segment, the management plane specifies the token value and exchanges the token value with the sender through a secure channel. The sender configures the token value to the UB device when importing the segment. When a UB device performs one-sided data-plane access, it carries a TokenValue. The receiver validates the token in the incoming one-sided request and proceeds with memory operations only after the validation passes.

The usage process is as follows:

1. Use local memory: Allocate a VA, and then register the segment via `urma_register_seg`.
2. Release local memory: Deregister the segment via `urma_unregister_seg`.
3. Use remote memory: Import remote memory via `urma_import_seg` to obtain `target_Segment` and UBA.
4. Release remote memory: Deregister the remote segment via `urma_unimport_seg`.

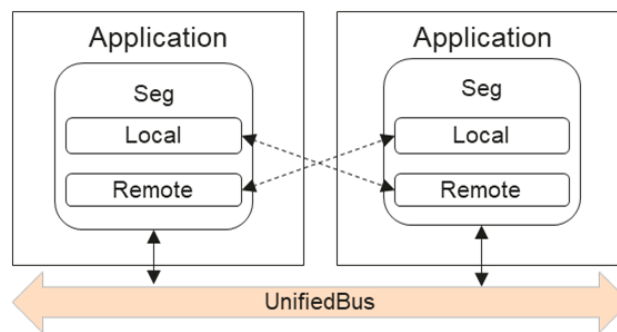


Figure 5-5 Segment usage process

5.3.3.3 Two-sided Operations

Message semantics provide a two-sided messaging service. Message semantics are asynchronous and non-blocking, the receiver shall receive messages proactively and read them before further processing. UMDK supports one-to-many message semantics. Messages can be sent from the same Jetty to different Jetties, which may be located on different remote nodes or processes. Two-sided operations support both contiguous and non-contiguous memory on the local and remote sides.

1. The send procedure is as follows:
 - (1) Call `urma_post_jetty_send_wr` to submit a send request through the JFS.
 - (2) Call `urma_poll_jfc` to poll for send requests. Only after the polling is complete can the send message buffer be reused (modified or released).
2. The receive procedure is as follows:
 - (1) Call `urma_post_jetty_rcv_wr` to submit a receive request, adding the local receive buffer to the JFR.

- (2) Call `urma_poll_jfc` to poll for receive requests. Only after the polling is complete can the message content be read from the receive buffer.

Note: To increase the throughput, the server can batch submit multiple receive requests. After a message is received, a new receive request is added to the JFR. Alternatively, when the number of receive requests in the JFR is less than the threshold, a new receive request is added to the JFR.

5.3.3.4 One-sided Operations

One-sided operations provide read and write semantics. These operations require knowledge of both local and remote memory addresses. During one-sided operations, only the local process performs the operation. No involvement or awareness is needed from the remote application. Two-sided operations are used to transmit control messages, while one-sided operations are used to transmit a large amount of data for data migration. One-sided operation buffers support local contiguous memory, local non-contiguous memory, and remote contiguous memory.

The read and write process is as follows:

1. Call `urma_post_jetty_send_wr` to submit a read or write request to a JFS that has been registered.
2. Call `urma_poll_jfc` to poll the JFC to check whether a CQE arrives. If the return value of `urma_poll_jfc` is greater than 0, a CQE has arrived, indicating that the read operation is complete. Only after the request is complete can the send message be reused (modified or released).

5.3.3.5 Atomic Operations

Atomic operations provide compare-and-swap (CAS) and fetch-and-add (FAA) semantics. These operations require knowledge of both local and remote memory addresses.

CAS atomic operations compare the original value of the remote address with the comparison value. If they are equal, the data at the remote address is updated to the new value; otherwise, the value remains unchanged.

FAA operations update the value at the remote address by adding the specified value to the original one, and return the original value.

The atomic operation process is the same as that described in Section 5.3.3.4.

5.3.3.6 Completion Records

All one-sided, two-sided, and atomic operations in UMDK are non-blocking. A success returned from an operation only indicates that the command has been placed into the send or receive queue. It does not necessarily mean that the operation has been completed. A completion record is used to describe the operation completion information. After the operation is complete, the hardware writes the completion record to the JFC completion queue. When the application polls the JFC, the hardware reads the

completion record from the completion queue and returns it to the application. URMA supports two modes for determining whether an operation is complete: polling and interrupt.

- Polling mode is used in low-latency scenarios. In this mode, the application proactively checks completion records to determine the execution status of operations and proceed accordingly. The application calls `urma_poll_jfc` to query completion records in polling mode. Polling is a non-blocking method for checking completion; if the completion queue is empty, no completion record will be returned.

Note 1: The application determines whether an operation is complete by checking the status fields of the completion records. If a fault occurs, the status field indicates the cause of the failure.

Note 2: The **completion_len** field represents the length of data successfully processed (such as the number of bytes sent or received). This field is only valid on the receive side, such as in `URMA_OPC_READ` or `URMA_OPC_RECV` operations.

Note 3: If the completion record is of the JFS type, data has been successfully sent, and the application can then modify or release the associated local buffer.

Note 4: If the completion record is of the JFR type, the application can read messages from the receive buffer.

Note 5: If `notify_data` is enabled, the completion record also carries the immediate data.

Note 6: The application associates a specific operation with its completion record through the **user_ctx** parameter (for example, the **user_ctx** parameter in the `urma_read` API).

- Interrupt mode is used in scenarios where communication is not frequent. In this mode, application threads remain in the sleep state while waiting for completion events, minimizing CPU overhead. The URMA wakes threads when a completion event arrives.

The interrupt process is as follows:

- (1) Enable the completion event by calling `urma_rearm_jfc`.
- (2) Submit the send operations (one-sided, two-sided, or atomic operations).
- (3) Call `urma_wait_jfc` to block and wait for a completion event. This function returns the JFC that generated the event and automatically disables it by default.
- (4) Repeatedly call `urma_poll_jfc` to retrieve all available completion records until none remain.
- (5) Go back to step (1) to re-enable the event.

5.3.4 Maintenance and Debugging

- `urma_admin`: The URMA framework exposes device resource attributes of different granularities through sysfs, including various Jetty attributes and port statuses. The `urma_admin` tool allows users to query device attributes and statuses directly from command lines and configure certain attributes, such as updating the EID or querying device information.

- `urma_perftest`: It is used in latency and bandwidth performance testing. It covers four types of semantics (send/receive, read, write, and atomic operations), each supporting both latency and bandwidth tests.

5.4 URPC

5.4.1 Overview

Within the servers, racks, and clusters of UB peer-to-peer architecture, although URMA provides memory semantic communication between CPUs, GPUs, and NPUs, different protocols exist at the functional layer. Mutual access between UBPU incurs protocol conversion overhead.

URPC defines the inter-UBPU access protocol. In UB, the relationship between any hardware components can be represented as function calls based on memory objects.

URPC has the following features:

- Peer-to-peer function calling: The CPUs are decentralized. Any type of UBPU can directly initiate function calls to other types of UBPU.
- Parameter passing by reference: Workers can initiate parameter data movement based on parameter data addresses.

5.4.2 Module Architecture

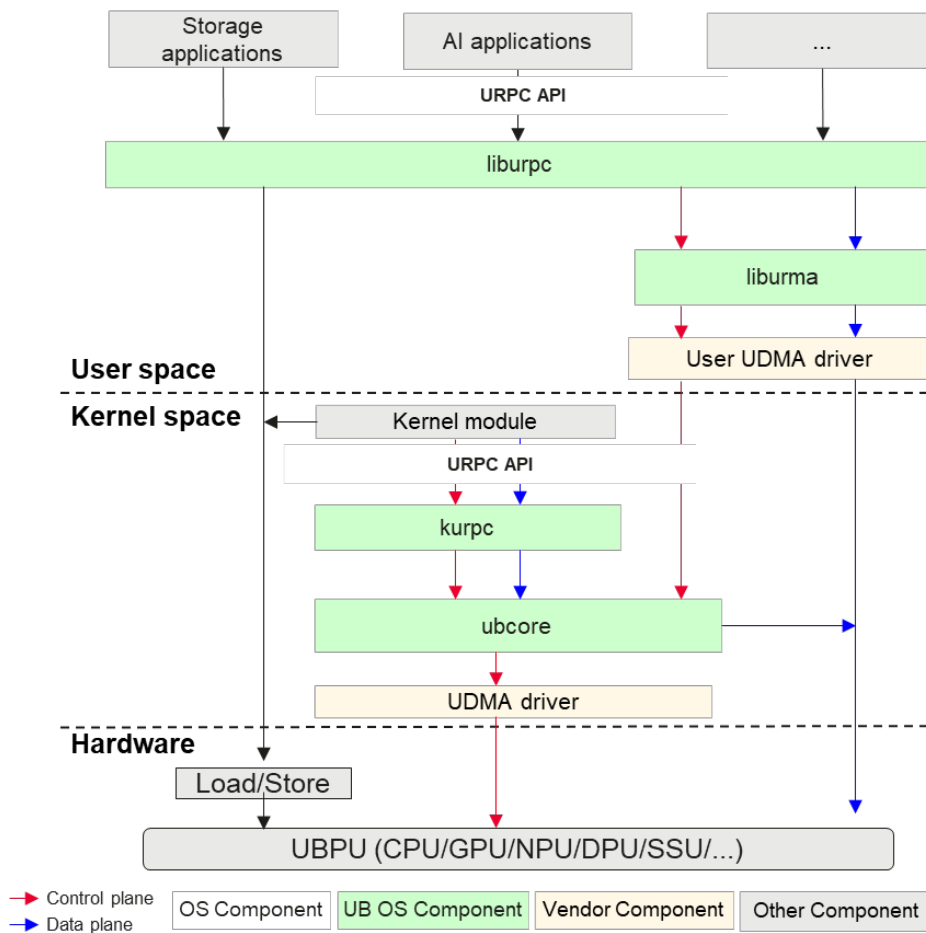


Figure 5-6 URPC module architecture

- **UDMA driver & User UDMA driver:** They are device drivers provided by hardware vendors. The UDMA driver operates in kernel space, while the User UDMA driver operates in user space, enabling UMDK integration in their respective spaces.
- **ubcore & liburma:** The ubcore module runs in kernel space, while liburma runs in user space. For details, see Section 5.3.
- **kurpc & liburpc:** They run in kernel and user spaces to provide an easy-to-use, unified, and efficient RPC mechanism for data center communication, offering northbound URPC programming interfaces for applications.

5.4.3 Usage Process

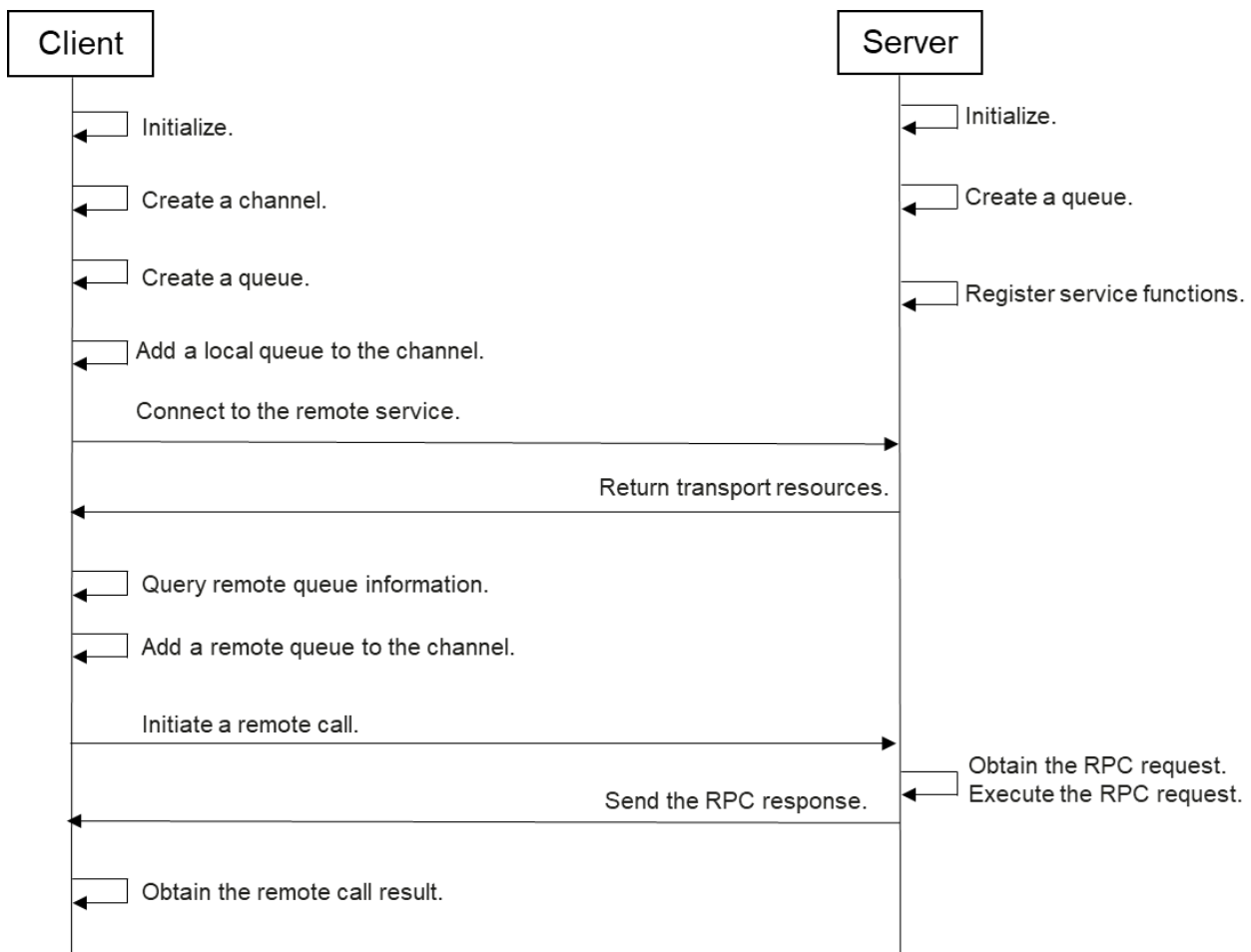


Figure 5-7 URPC usage process

1. `urpc_init`: The client and server initialize essential resources.
2. `urpc_func_register`: The server registers custom functions to obtain their function IDs.
3. `urpc_channel_create`: The client creates a channel.
4. `urpc_queue_create`: Both the client and server create queues and initialize transport-layer resources (Jetty or memory) based on the specified mode.
5. `urpc_server_start`: The server finishes resource initialization and starts the listening thread to accept client attachment.
6. `urpc_channel_queue_add`: The client adds the local queue to a channel.
7. `urpc_channel_server_attach`: The client mounts the channel to the server and shares its local queue information with the server. On the server side, a connection is established for the client-side queue. Then the server sends the queue and function information to the client.
8. `urpc_channel_queue_query`: The client queries the remote queue information exchanged from the server.
9. `urpc_channel_queue_add`: The client adds the remote queue to the channel, completing the linkage to the queue on the server.
10. `urpc_func_call`: The application on the client side calls this API to initiate a function call.

11. `urpc_fun_poll`: The worker calls this API to poll and obtain the request (parameters), `func_id`, and other required information for function execution.
12. `urpc_func_exec`: The worker calls this API to execute the registered custom function based on the `func_id` and request parameters, returning a response.
13. `urpc_func_return`: The worker calls this API to return the response to the client.
14. `urpc_fun_poll`: The client application calls this API to retrieve the response and release the corresponding request, process the response, and free the request resources.

5.4.3.1 Channel Creation

A URPC Channel is an abstract channel that ensures lossless transmission of URPC

Request/Ack/Response messages between a client and a server. The client sends URPC Requests to the server through the URPC Channel and reads the corresponding URPC ACK/Response returned by the server from the same channel.

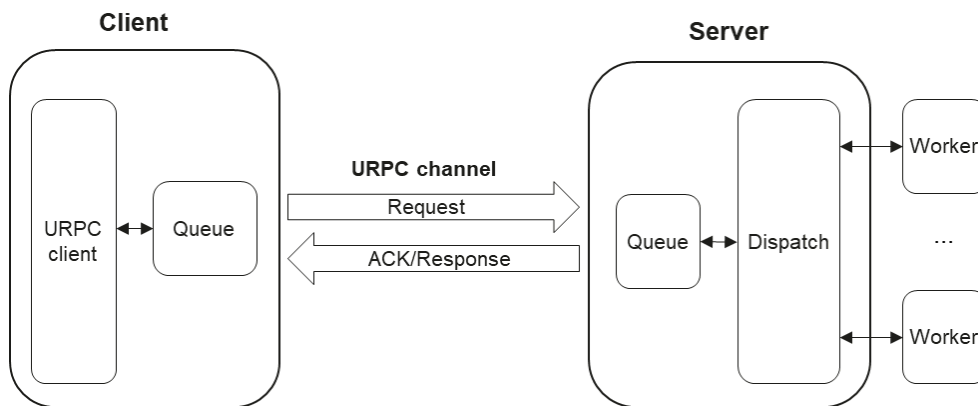


Figure 5-8 Channel logic

1. Call `urpc_channel_create` to create a channel.
2. Call `urpc_channel_server_attach` to connect to a remote server.
3. Call `urpc_channel_queue_add` to add the local and remote queues to the channel.

5.4.3.2 Queue Creation

A queue is a set of transmission resources of the source or destination. It can be mapped to different Jetties in different scenarios. Both the client and server call `urpc_queue_create` to create a queue.

5.4.3.3 Function Registration

The server calls `urpc_func_register` to register the function.

5.4.3.4 Function Calling

1. Call `urpc_func_call` to initiate a function.
2. Call `urpc_func_poll` to get the execution result.

5.4.3.5 Function Execution

1. Call `urpc_func_poll` to receive requests.
2. Call `urpc_func_exec` to execute a registered function and obtain the execution result.

5.4.3.6 Function Return

Call `urpc_func_return` to return the execution result to the initiator.

5.4.4 Maintenance and Debugging

`urpc_admin`: URPC records message statistics and memory usage, and dynamically enables segmental delay measurement. `urpc_admin` can be used to query information and locate faults.

5.5 UMS

5.5.1 Overview

UMS supports interconnection with the socket abstraction layer to provide socket compatibility, enabling existing socket interfaces to leverage the high-performance UB communication for enhanced performance.

5.5.2 Module Architecture

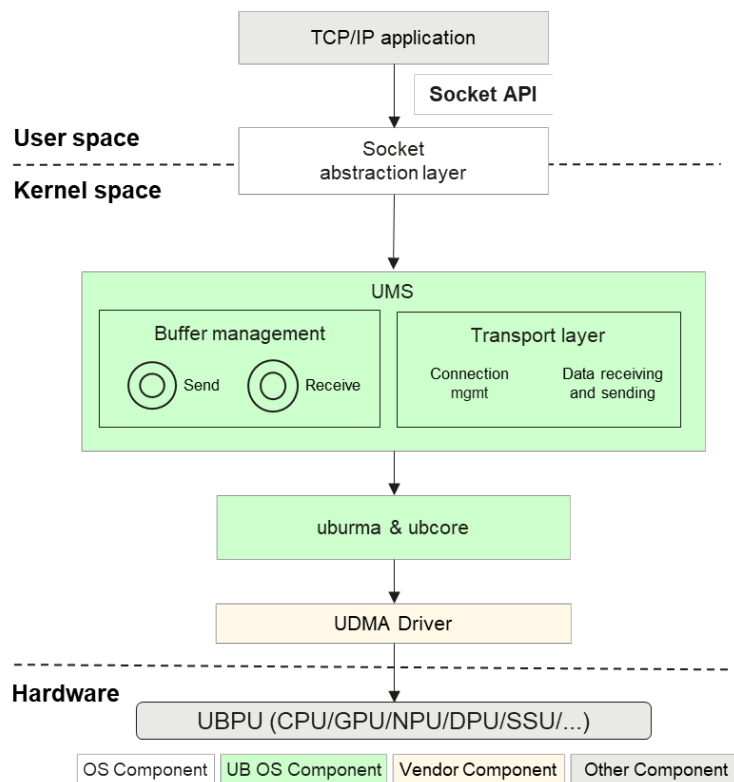


Figure 5-9 UMS module architecture

- **UMS:** It provides socket-compatible interfaces and accelerates communication performance using the URMA Jetty asynchronous communication or load/store semantics of UB in kernel space.
- **uburma & ubcore:** For details about URMA components, see Section 5.3.

5.5.3 Usage Process

An application can use UMS in either of the following ways:

- Change the protocol family type when the application applies for a socket.

```
socketfd = socket(AF_INET, SOCK_STREAM, 0) --> socketfd = socket(AF_UB, SOCK_STREAM, 0)
```

- Add **ums_run** to the startup script.

```
ums_run gaussdb -D /data2/lite/data -M primary
```

5.5.4 Maintenance and Debugging

ums_admin: It is used to query communication link status information, including fields such as **link_id**, **link_state**, **sndbuf_cnt**, and **rmb_cnt**.

6 Virtualization

6.1 Overview

This chapter describes the virtualization implementation of UB, UB Entity, Bus Controller, and UMMU. In cloud computing scenarios, hardware resources on a physical machine are isolated and provisioned to VMs through virtualization, significantly enhancing resource utilization. As a next-generation high-speed interconnect, UB natively supports virtualization.

Mainstream device virtualization technologies are as follows:

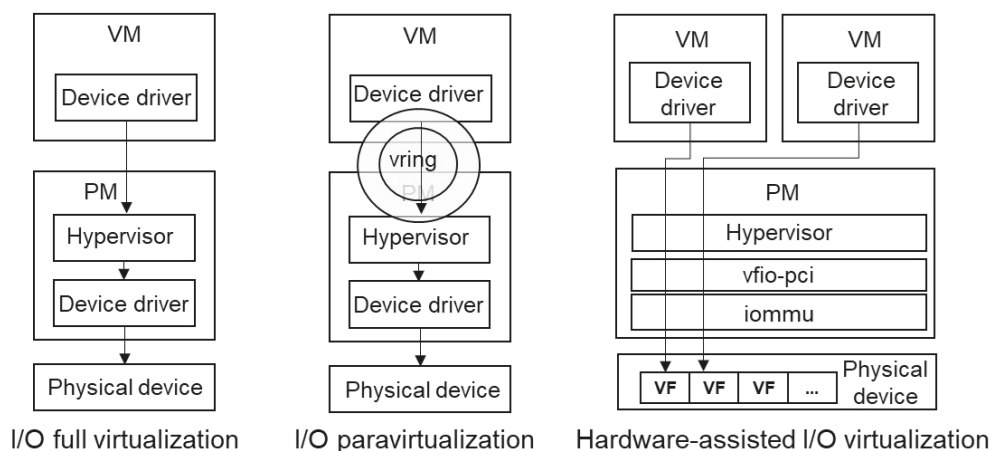


Figure 6-1 Mainstream device virtualization technologies

Under traditional architectures, each server connects to the Top of Rack (TOR) switch through its own NIC, meaning its maximum supported bandwidth is fixed. This can lead to several issues in real-world applications, as illustrated below.

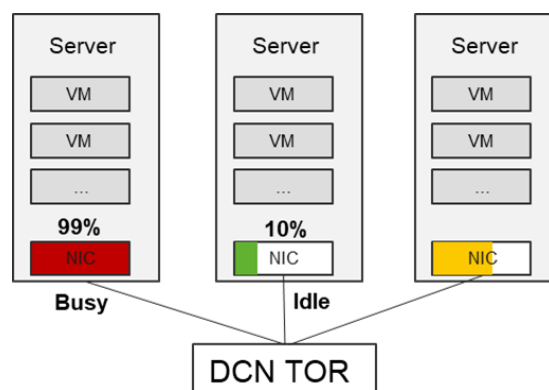


Figure 6-2 NIC traffic diagram in traditional architectures

- Low bandwidth utilization during off-peak periods: Service deployment typically estimates a theoretical peak bandwidth requirement, but in practice, workloads often remain far below this threshold. The bandwidth usage increases only during peak hours. This results in

underutilized bandwidth and wasted reserved capacity. To address this, public cloud vendors apply virtualization and bandwidth oversubscription techniques to improve overall physical bandwidth utilization.

- Bandwidth bottleneck during peak periods: Under heavy workloads, bandwidth pressure increases and potentially leads to insufficient bandwidth. Especially in oversubscription scenarios, the total committed bandwidth across all VMs exceeds the maximum bandwidth capacity of the NIC on the physical machine.
- Uneven bandwidth pressure across servers: Since each server has a fixed NIC bandwidth, variations in workload distribution can cause imbalanced network pressure across servers. This makes it difficult to ensure consistent network service quality.

A UB Entity acts as an isolated functional unit within a UB device. Virtualization in UB supports direct device assignment to VMs, delivering native-level performance. Compared to traditional hardware-assisted I/O virtualization, UB device virtualization not only supports passthrough to local UB devices but also to pooled UB devices.

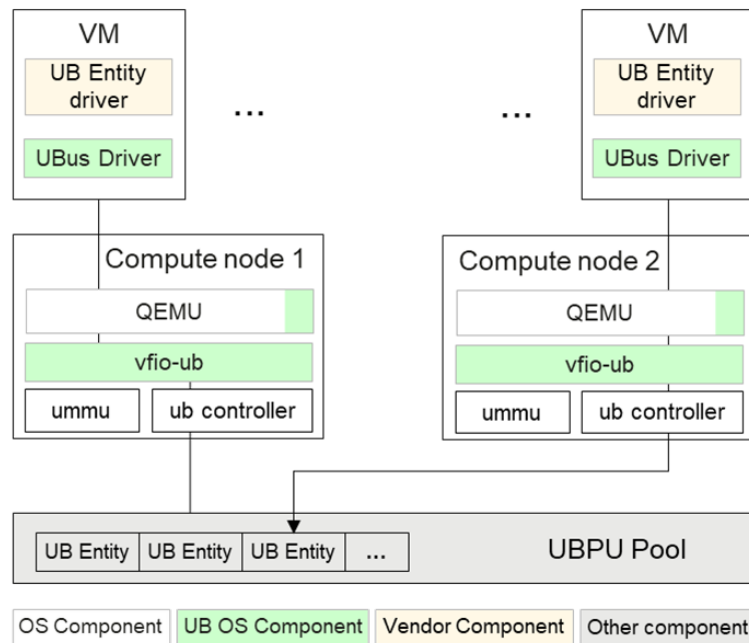


Figure 6-3 UB device virtualization

The preceding problems can be solved by pooling NIC and DPU resources. All device resources are aggregated in a shared device pool, allowing servers to dynamically request NIC resources based on their actual network load. This approach improves overall resource utilization and helps prevent network bandwidth bottlenecks.

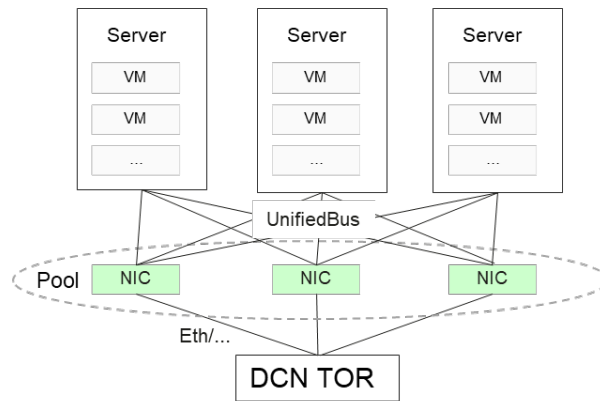


Figure 6-4 NIC traffic diagram in UB

6.2 Functional Architecture

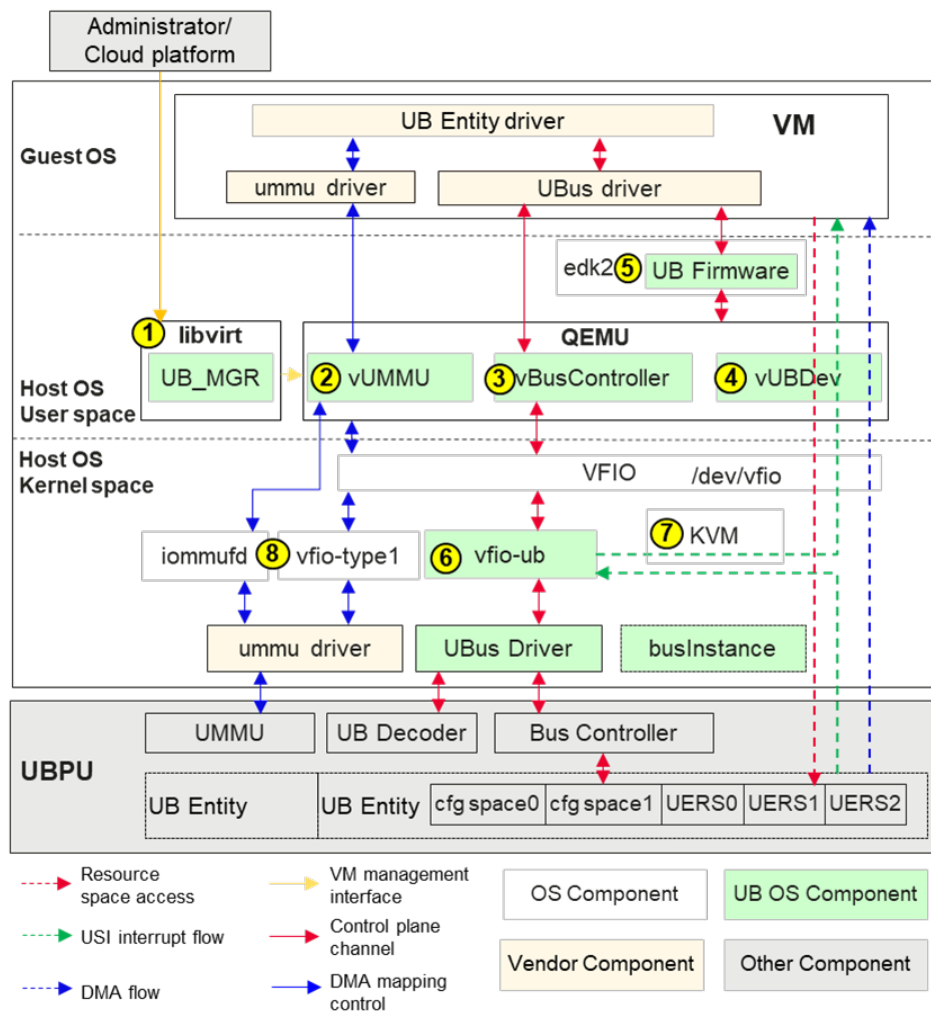


Figure 6-5 UB virtualization architecture

- ① **UB_MGR**: It creates and manages UB VMs using libvirt XML configuration files.
- ② **vUMMU**: It emulates UMMU for the VMs, enables the UMMU driver to create stage-1 page tables (GVA to GPA), synchronizes their addresses with the physical UMMU to complete the UMMU page table configuration.
- ③ **vBusController**: It simulates a Bus Controller for the VMs, provides a virtual UB entry, and enables the UB drivers inside each VM to perform UB device topology scanning, discovery, and access configuration through the Bus Controller.
- ④ **vUBDev**: It simulates UB device models for VMs, providing complete UB configuration, resource, and virtual topology spaces, to map the resource space.
- ⑤ **UB Firmware**: It reports Bus Controller information, UMMU information, allocatable UB resources, and the mapping between the UMMU and the interrupt controller to the guest OS.
- ⑥ **vfio-ub**: It allows user-space device management and exposes UB devices to user-space QEMU.
- ⑦ **KVM**: It remaps UB signaled interrupt (USI) and supports virtual USI injection or passthrough.
- ⑧ **vfio-type1/iommufd**: It provides APIs for managing I/O page tables in user space for UMMU configuration.

6.3 Usage Process

UBPRM registers pooled device resources to the target VM. During this process, a corresponding BusInstance is created on the host to complete the device registration.

After the UB driver completes device registration and initialization, the list of UB device resources available on the host can be retrieved using the **lsub** command. Selected devices can then be directly passed through to VMs using XML configuration.

```
# lsub
<00002> Class <0200>: Huawei Technologies Co., Ltd. Device <00e0fc>:<a001>
<00004> Class <0a00>: Huawei Technologies Co., Ltd. Device <00e0fc>:<a005>
<00001> Class <0000>: Huawei Technologies Co., Ltd. Device <00e0fc>:<a000>
<00003> Class <0200>: Huawei Technologies Co., Ltd. Device <00e0fc>:<a003>
```

The BusInstance can be queried using lsub or by running the **cat/sys/bus/ub/instance** command.

```
# lsub -b
BusInstance show format: guid type eid upi
00e0fc-0-0-a000-0000-000000000001 Static_Server    00002 7fff
00e0fc-0-0-a000-0000-000000000012 Dynamic_Server  00007 7fff
# cat /sys/bus/ub/instance
```

```
count 0x2, from 0x0, show 0x2
guid:00e0fc-0-0-a000-0000-0000000001 type:0 eid:00002 upi:7fff
guid:00e0fc-0-0-a000-0000-0000000012 type:2 eid:00007 upi:7fff
```

The dynamic-type BusInstance is created specifically for VMs.

After a device is registered, the user can query the sysfs instance to confirm the BusInstance bound to the device.

```
#cat /sys/bus/ub/devices/00002/instance
0x00007
```

Configure UB passthrough devices using an XML file:

1. Configure a UB Controller for the VM.

The libvirt XML has been extended to support the creation of Bus Controller and enable UB passthrough. To use UB devices, VMs shall support UB. Therefore, a Bus Controller shall be provided. A new Bus Controller can be defined with the **controller** label. The following is an example of defining a Bus Controller:

```
<controller type='ub'>
  <ports num='10'/>
  <source>
    <businstance guid='00e-0fc-0-0-a000-0000-0000000-0000000012'/>
  </source>
</controller>
```

ports num specifies the maximum number of ports supported by the Bus Controller configuration. Each UB device occupies one port. Because the UBus Driver uses an 8-bit port index, the Bus Controller can support up to 256 ports. This parameter supports automatic configuration. If the **ports** tag is not specified, the default value is 128.

businstance guid indicates the GUID of a dynamic BusInstance created by UBFM for a VM. The GUID can be queried using the lsub tool.

```
# lsub -b
BusInstance show format: guid type eid upi
00e0fc-0-0-a000-0000-0000000001 Static_Server 00002 7fff
00e0fc-0-0-a000-0000-0000000012 Dynamic_Server 00007 7fff
```

2. Configure UMMU for the VM.

Configure **iommufds** and **ummu** for the VM:

```
<iommufds>1</iommufds>
<devices>
  <iommu model='ummu'>
</iommu>
```

```
</devices>
```

3. Configure UB device passthrough for the VM.

A UB passthrough device is defined using the **hostdev** tag. The simplest configuration is as follows:

```
<hostdev mode='subsystem' type='ub' iommufd='1'>
  <driver name='vfio'>
    <source>
      <address guid="00-e0fc-2-0-a003-0200-000000-00000000001"/>
    </source>
  </hostdev>
```

The **source** information specifies the physical UB device on the host. **guid** contains the vendor information, which can be queried under the path to the corresponding UB device to be passed through on the host, for example:

```
# cat /sys/bus/ub/devices/00001/guid
00-e0fc-8-0-a000-0000-000000-00000000001
# cat /sys/bus/ub/devices/00002/guid
00-e0fc-2-0-a001-0200-000000-00000000001
# cat /sys/bus/ub/devices/00003/guid
00-e0fc-2-0-a003-0200-000000-00000000001
# cat /sys/bus/ub/devices/00004/guid
00-e0fc-2-0-a005-0a00-000000-00000000001
```

Because the UMMU permission table requires contiguous memory, the VM shall also be configured to use huge pages:

```
<memoryBacking>
  <hugepages>
    <page size='2048' unit='KiB'>
  </hugepages>
</memoryBacking>
```

After the configuration is complete, the VM starts up in the same way as a traditional VM.

6.4 Maintenance and Debugging

A series of hmp debug commands is added to QEMU to analyze UB and UB device information on VMs. Run the **qemu hmp** command through libvirt:

```
# virsh qemu-monitor-command --help
NAME
    qemu-monitor-command - qemu Monitor Command
SYNOPSIS
```

qemu-monitor-command <domain> [--hmp] [--pretty] [--return-value] [--cmd] <string>...

DESCRIPTION

qemu Monitor Command

OPTIONS

- [--domain] <string> domain name, id or uuid
- hmp command is in human monitor protocol
- pretty pretty-print any qemu monitor protocol output
- return-value extract the value of the 'return' key from the returned string
- [--cmd] <string> command

Example: **virsh qemu-monitor-command my_vm --hmp my_hmp_cmd**

- 1. Query UB information.

Command: **info ub**

The **info ub** lists the brief information about all UB devices in the current VM. Example:

```
# virsh qemu-monitor-command vm_xxx --hmp info ub
```

ID	Name	Bus	Eid	CNA	Ueldx	Guid	Type	PortNum
myvfioub1	vfiou-b	0	2	0	0	00-e0fc-2-0-a001-0008-000000-1234567890	2	1
myubc	ubc	0	1	0	0	00-e0fc-8-0-1111-0000-000000-0000000123	8	3

- 2. Query UB device information.

Command: **info ub [id]**

This command provides the details about the UB device with a specified ID. **id** indicates the ID of the UB device to be queried. The user can run the **info ub** command to obtain the IDs of all UB devices. Example:

```
# virsh qemu-monitor-command vm_xxx --hmp info ub myubc
```

id	myubc
dev_type	8 type_controller
cluster_mode	0
fm_deployment	0
config	0xfffc275c7010
config_size	2147483648
name	ubc
eid	1
config0:total ports	3
config0:total UEs	1
config0:cap_bitmap	0x00000004

...	...		
MR msgq_reg_mem name	ub-bus-controller		
MR io_mmio name	UB_MMIO		
hi_msgq_info sq addr	gpa 0x0	hva 0x0	
hi_msgq_info cq addr	gpa 0x0	hva 0x0	
hi_msgq_info rq addr	gpa 0x0	hva 0x0	

3. Query UB device configuration space information.

Command: **info ub-config id offset len**

This command displays the configuration space information at the offset of a specified UB device. **ubdev_id** indicates the ID of the UB device to be queried, **offset** indicates the offset address of the configuration space to be queried, and **len** indicates the configuration space length. Example:

```
# virsh qemu-monitor-command vm_XXX --hmp info ub-config myvfioub1 0 1924
```

	0x0	0x4	0x8	0xC
0x0	00307830	00010001	00000000	00000000
0x10	00000000	00000000	00000000	00000000
...				
0x30	00000000	00000000	34567890	00000012
0x40	a0010008	00e0fc20	00000000	00000000
0x50	00000000	00000000	00000000	00000000
...				
0x780	00000000			

7 RAS

7.1 Overview

This chapter describes the Reliability, Availability, and Serviceability (RAS) features of the OS within UB, covering the detection, isolation, and recovery capabilities for UB faults, as well as handling recommendations for related components such as applications, UBFM, UBPRM, and UBMM.

Table 7-1 RAS features

RAS Feature	Description
UB device fault isolation and recovery	Fault detection and recovery of UB Entities and Bus Controllers.
Remote memory fault isolation and recovery	Fault detection and recovery when accessing remote memory that is borrowed or shared.
Communication fault isolation and recovery	Fault detection and recovery of URMA communication.
Out of memory (OOM) prevention and recovery	Prevention and mitigation of OOM using the UB memory borrowing capability.
Node fault isolation and recovery	When a node panic occurs, memory migration and other strategies are applied to prevent fault propagation and ensure service continuity in the memory pool.

Note 1: Local RAS faults in the UBPU hardware modules are handled in the traditional single-node manner and reported by the BIOS to BMC and the OS. These faults are not elaborated in this document.

Note 2: In Section 10.6.2 of *UnifiedBus™ (UB) Base Specification 2.0*, errors are classified into three types:

In the case of a class-A error, the physical topology of the UB Fabric remains intact. The error can be traced to a specific transaction and does not affect the execution of other transactions.

In the case of a class-B error, the physical topology of the UB Fabric remains intact. The error can be traced to a specific UB Entity, TP channel, or Jetty (UB Entity-level, TP channel-level, or Jetty-level error) rather than a particular transaction. All resources of the error unit need to be processed.

Class-C errors occur on device- or port-level public resources.

7.2 Functional Architecture

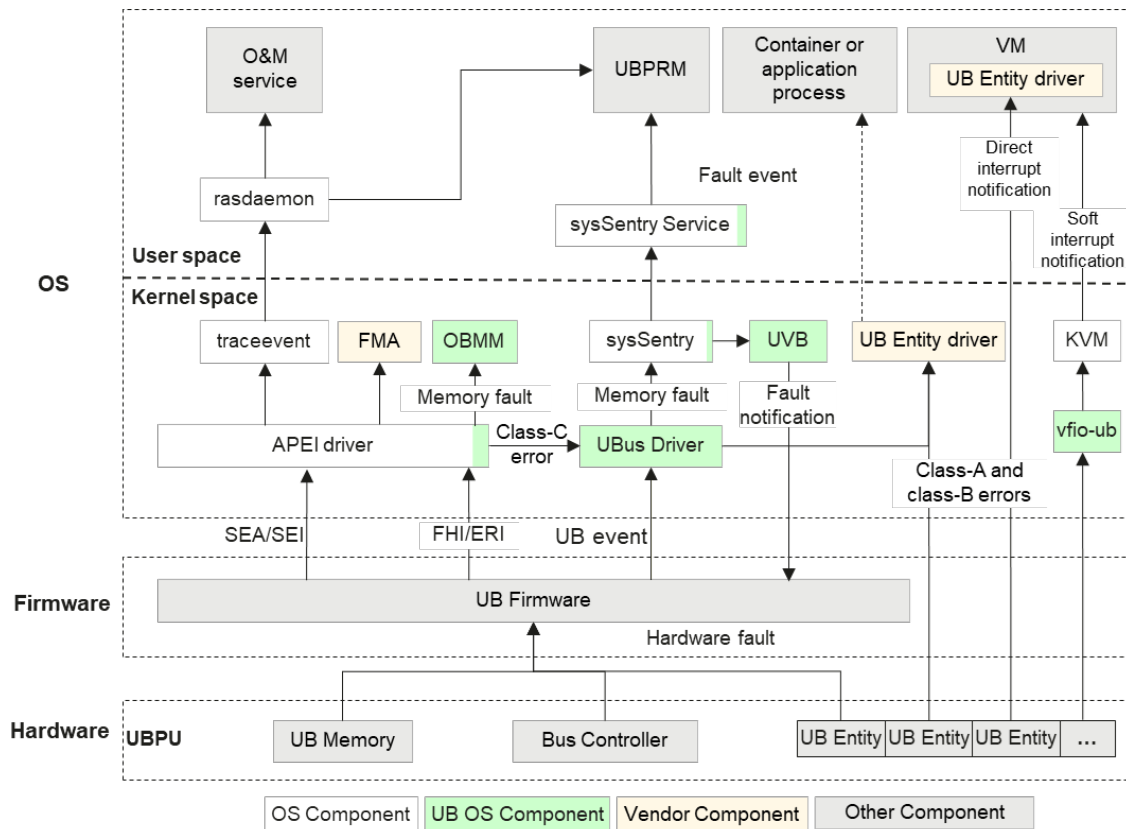


Figure 7-1 UB RAS framework

The components and their functions are as follows:

- Hardware and firmware
 - **UB Memory, Bus Controller, and UB Entity:** Errors in these hardware components may occur during operation. The hardware reports the corresponding error to UB Firmware.
 - **UB Entity:** If a service error (class-A or class-B) occurs, the error is directly reported to the corresponding UB Entity driver.
 - **UB Firmware:** It receives hardware error events and forwards them to the OS.
- OS components
 - **ACPI Platform Error Interfaces (APEI) driver:** Hardware faults are reported according to the APEI specification and processed by the APEI driver.
 - **traceevent:** The kernel uses this mechanism to record various events. When a hardware error occurs, the hardware reports the non_standard_event event through the APEI driver.
 - **rasdaemon:** It listens to events reported by EDAC and traceevent, records related logs, and provides the logs for upper-layer services to monitor and process.
- UB components
 - **UBus Driver:** It receives class-C errors reported by UB Firmware through APEI and notifies the corresponding module for processing.

- **sysSentry/sysSentry Service:** When emergency events such as OOM, panic, or reboot occur, it blocks the corresponding events and reports them to UBPRM to prevent data loss or service interruption. It is also responsible for unified management of UB event reports.
- **OBMM:** It handles memory-related errors. When a memory fault occurs on the Home side, OBMM marks the faulty memory to prevent further access or reuse.
- **UVB:** When a fault occurs on the Home-side node, it broadcasts the fault information to other nodes for coordinated handling.
- **vfio-ub:** When the interrupt soft notification mechanism is used, if a device fault occurs, the fault is reported to vfio-ub, which then notifies the VM through KVM.
- Vendor components
 - **FMA:** It serves as the memory fault isolation notification chain. Services can use this chain to bypass the OS's default memory fault isolation logic and perform customized fault handling as needed.
 - **UB Entity driver:** It handles various UB driver faults and can implement customized handling logic based on specific service requirements.

7.3 Fault Detection

7.3.1 UB Device Fault Detection

UB device faults refer to faults of UB Entities, Bus Controllers, and other devices. The fault handling methods vary according to the device type.

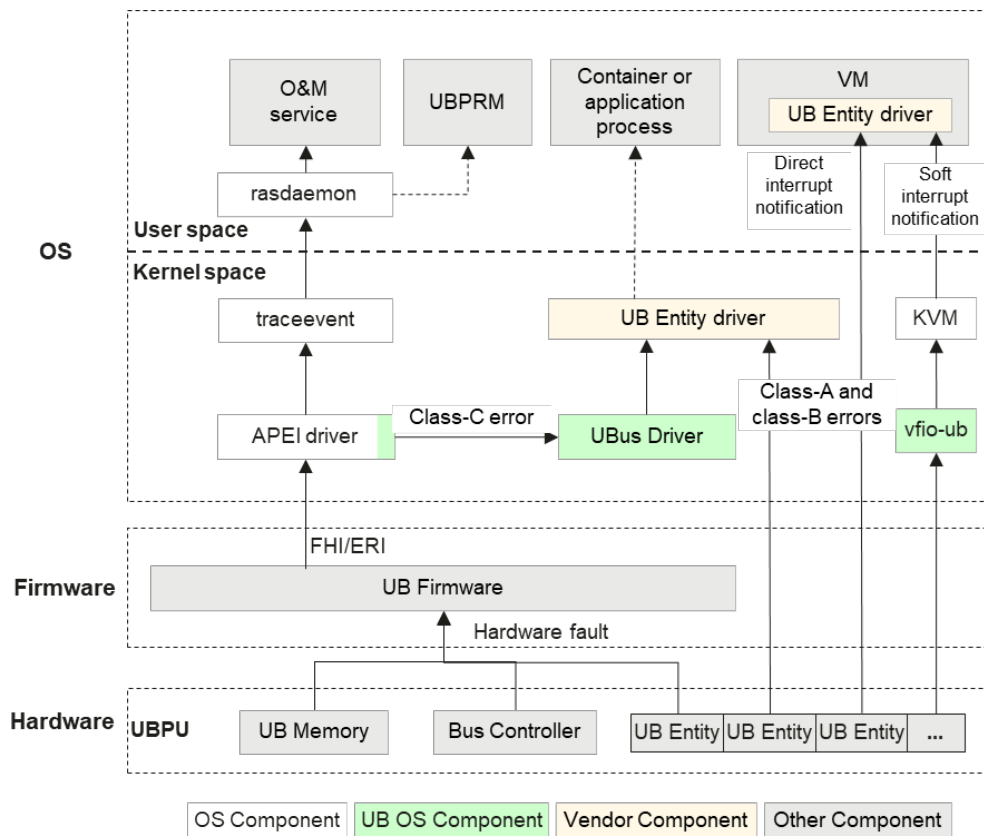


Figure 7-2 UB device fault detection framework

UB device errors are classified into two types:

- Class-A and class-B errors
 - (1) For UB Entity service faults, UB Entity driver obtains the fault status by querying the status registers in the UB Entity's configuration space. The corresponding component using the UB Entity then passes the fault information to the service layer.
 - (2) In virtualization scenarios, when a UB Entity interrupt is passed through to a VM, the UB Entity driver inside the VM receives it. If the UB Entity interrupt requires host participation for soft notification, the interrupt is first received by vfio-ub on the host, which then notifies the VM through KVM.
- Class-C errors
 - (1) UB Firmware triggers the FHI/ERI interrupts.
 - (2) The OS kernel reports traceevent error events through the APEI driver and notifies UBus Driver.
 - (3) The OS kernel reports the fault to rasdaemon using traceevent.

7.3.2 Remote Memory Fault Detection

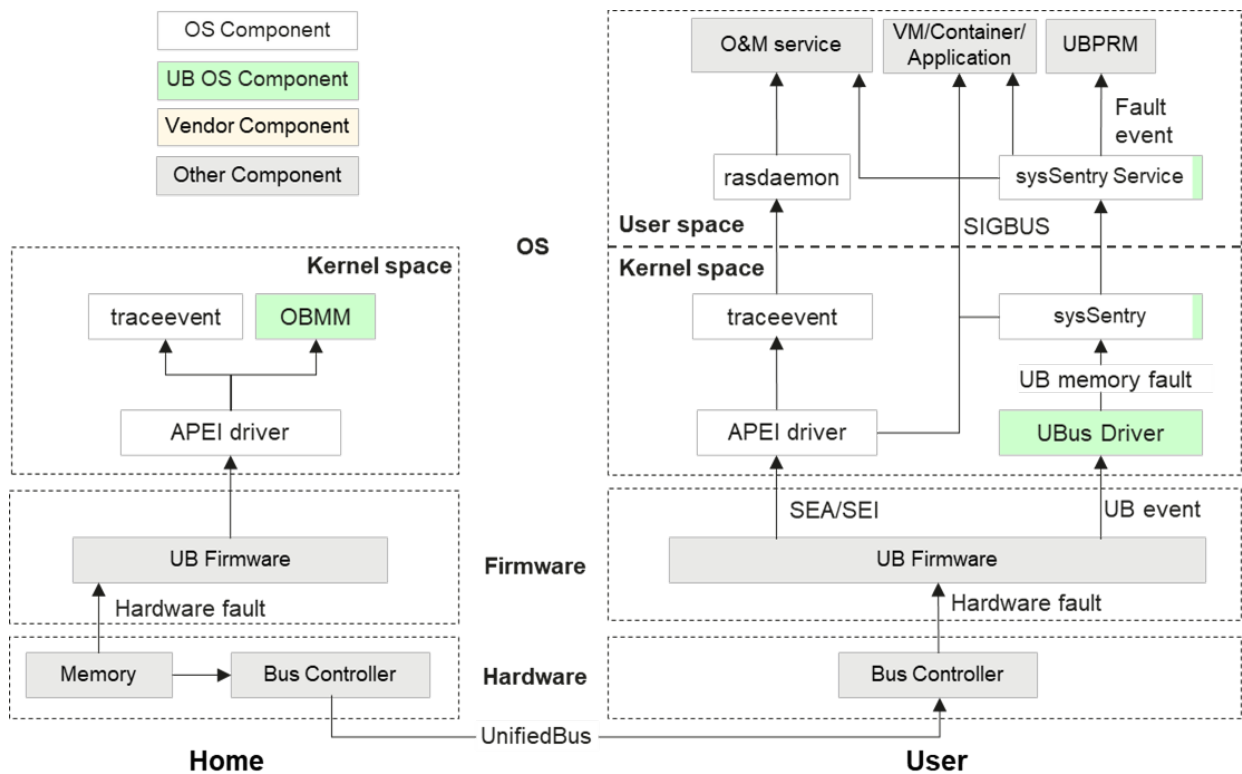


Figure 7-3 Remote memory fault detection framework

In scenarios involving borrowed or shared memory, faults shall be detected and reported by both the User and Home sides. The process is divided into the following cases:

- Home-side memory fault reporting
 - (1) The Home memory controller reports the memory fault to UB Firmware.
 - (2) UB Firmware reports the fault to the OS.
 - (3) The APEI driver in the OS kernel notifies traceevent and OBMM.
- User-side read fault on remote memory
 - (1) UB Firmware triggers SEA/SEI interrupts.
 - (2) The OS kernel APEI driver reports the fault to traceevent.
- User-side write fault on remote memory
 - (1) UB Firmware reports the fault as a UB event to UBus Driver.
 - (2) UBus Driver reports the fault to sysSentry.
 - (3) sysSentry forwards the message to sysSentry Service.
 - (4) sysSentry Service reports the fault information to UBPRM and the O&M service.

7.3.3 Communication Fault Detection

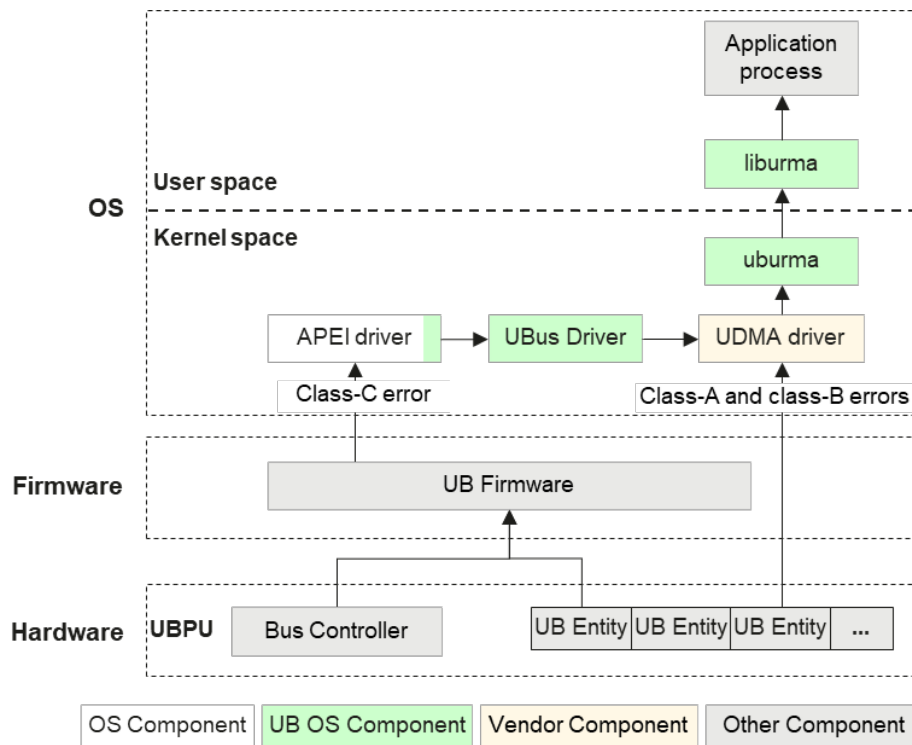


Figure 7-4 Communication fault reporting process

- Refer to Section 7.3.1 for UDMA fault details.
- Communication control plane faults (class-B protocol faults): When the hardware encounters an unprocessable WR—for example, accessing memory beyond local or remote permissions, Jetty or JFC overflow, driver uninstallation, or abnormal port status—the UDMA driver retrieves the exception event by querying the status registers in the UDMA configuration space. The UDMA driver then reports the fault upward through uburma and liburma, ultimately notifying the service process.
- Communication data plane faults (class-A protocol faults): When the data plane experiences packet transmission anomalies—such as ACK timeout or unsupported operation modes—the fault notification follows the same process as notification of the communication control plane faults.

7.3.4 OOM Detection

In memory borrowing scenarios, UBPRM periodically monitors memory watermarks. This watermark is configurable and can trigger memory reclamation or memory borrowing. However, if memory usage spikes rapidly within a short period, the UBPRM detection mechanism may not catch the change in time, potentially causing processes on service nodes to be stopped or nodes to restart, leading to service interruptions. The OOM prevention mechanism addresses this by reporting OOM events to UBPRM, triggering emergency borrowing. The OOM prevention progress is as follows:

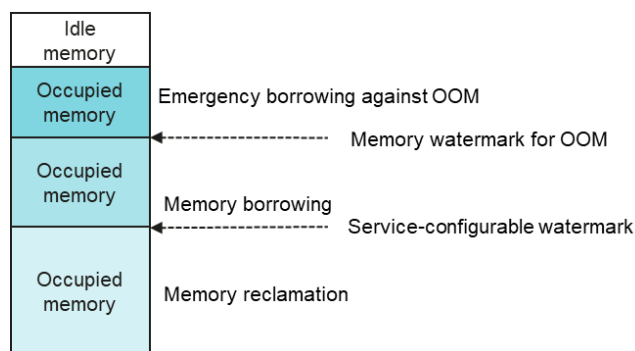


Figure 7-5 OOM prevention processing policy

When OOM emergency borrowing is triggered, the notification process is as follows:

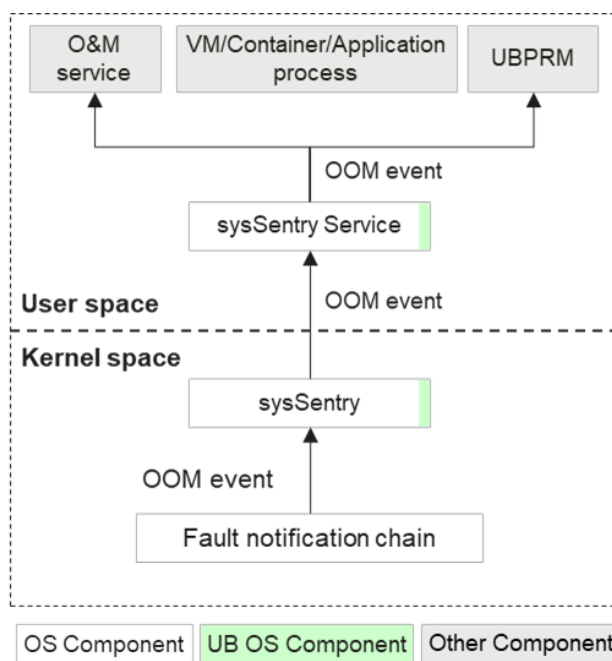


Figure 7-6 OOM emergency borrowing notification process

1. The node where the OOM fault occurs reports the event to sysSentry via the OOM event notification chain.
2. sysSentry reports the OOM event to sysSentry Service.
3. sysSentry Service reports the OOM event to UBPRM or the O&M service for handling.

7.3.5 Node Fault Detection

When a Home-side node experiences a panic or reboot, the memory it has lent becomes inaccessible, which may impact application processes on the User side. In this situation, the data stored in the Home-side memory shall be migrated to ensure service continuity.

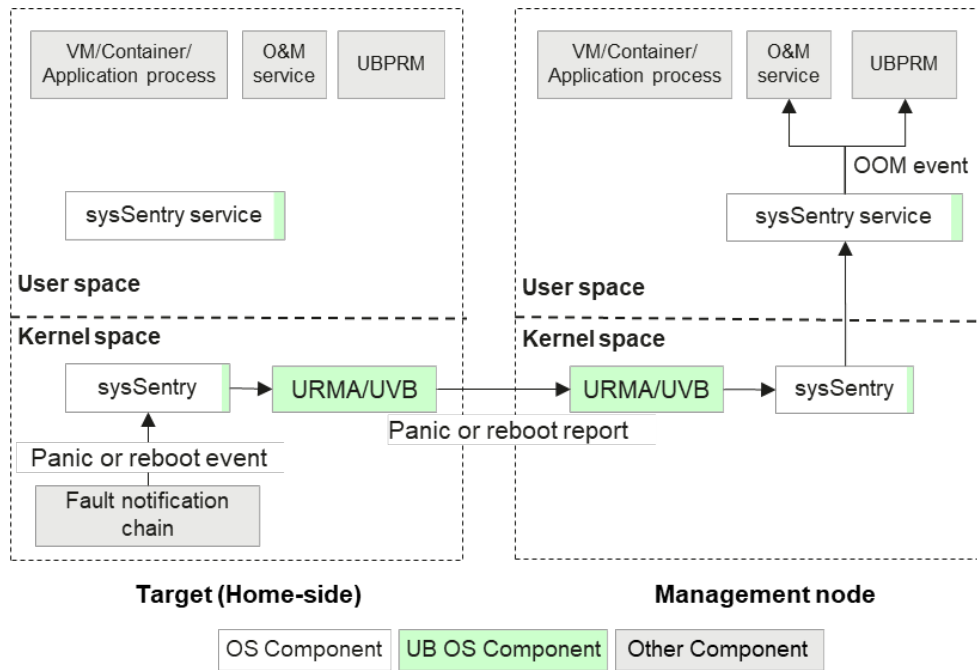


Figure 7-7 Node fault detection framework

The notification process is as follows:

1. During the Home-side node kernel fault process (for example, a panic, reboot, or shutdown), sysSentry is notified of the imminent reset and restart.
2. sysSentry broadcasts the fault event through the URMA/UVB channels.
3. The fault event is transmitted from the Home-side node's URMA/UVB channels to the URMA/UVB channels of the management node.
4. sysSentry on the management node receives the fault event information and notifies the management node UBPRM through sysSentry Service.

7.4 Fault Isolation and Recovery

7.4.1 UB Device Fault Isolation and Recovery

1. For class-A and class-B faults, the corresponding driver or service process handles the fault directly, for example, by releasing affected resources or retrying operations.
2. For correctable class-C errors, they can recover automatically, and UBus Driver only logs the event without further action. For uncorrectable errors, the corresponding UB Entity driver is informed to take appropriate measures, for example, resetting a faulty port and logging the event.
3. UBPRM subscribes to non_standard_event events through system traceevent and processes the fault events.
4. The O&M platform monitors and handles faults by checking logs of events reported by rasdaemon.

7.4.2 Remote Memory Fault Isolation and Recovery

Applications or UBPRM can subscribe to sysSentry remote memory access failure events, receive the corresponding fault information, and isolate the faulty memory or restore the applications based on the received fault information.

- Home-side memory fault isolation and recovery
 - (1) The APEI driver isolates the faulty memory.
 - (2) OBMM marks the faulty memory as unavailable for borrowing.
- User-side remote memory read fault isolation and recovery
 - (1) When remote memory is bound to a NUMA node, faulty memory is isolated by the APEI driver, which sends SIGBUS to the application processes that are using the memory.
 - (2) When remote memory is bound to a character device, sysSentry Service notifies subscribers—including UBPRM, O&M service, and application processes—via events. Subscribers can take appropriate actions, such as isolating the memory or reallocating it.
 - (3) The O&M service and UBPRM monitor rasdaemon logs and perform necessary fault handling.
- User-side remote memory write fault isolation and recovery
 - (1) sysSentry sends SIGBUS to application processes that are using the memory.
 - (2) sysSentry Service notifies subscribers—including UBPRM, O&M service, and application processes—via events. Subscribers can take appropriate actions, such as isolating the memory or requesting memory again.

7.4.3 Communication Fault Isolation and Recovery

- Communication control-plane fault recovery
 - (1) Applications call the `urma_get_async_event` interface to obtain exception events.
 - (2) Applications classify and handle the events based on the event type (see `urma_async_event_type`), for example, printing the logs.
 - (3) Applications call the `urma_ack_async_event` interface to notify URMA that the exception event has been processed.
- Communication data-plane fault recovery
 - (1) Applications call the `urma_poll_jfc` interface to retrieve anomaly status.
 - (2) Applications classify and handle the events based on the event type (see `urma_cr_status`), for example, printing the logs.

7.4.4 OOM Prevention, Isolation, and Recovery

UBPRM can subscribe to OOM-type events from sysSentry. When an OOM event occurs, UBPRM receives the notification from the sysSentry service and performs corresponding handling. The recommended handling procedure is as follows:

1. UBPRM borrows memory from other nodes and imports it to the OS of the affected service node.
2. The OS attempts to allocate memory during an OOM event; if successful, it returns memory to the corresponding application; otherwise, the application is terminated. If OOM occurs during a kernel-space allocation, the system enters a panic flow and reboots.

7.4.5 Node Fault Isolation and Recovery

UBPRM can configure sysSentry via sentryctl to intercept panic/reboot events and set necessary parameters for cross-node communication. It subscribes to OS panic and reboot events to monitor such events.

Upon receiving a fault notification from the sysSentry Service, UBPRM performs isolation and recovery measures, for example, migrating Home-side memory to other nodes and releasing the borrowed memory from the faulty node to ensure service continuity for services that are using pooled memory.

After handling the fault, the system recovers according to the following procedure:

1. UBPRM notifies sysSentry Service of the fault handling result.
2. UBPRM transmits the result through sysSentry and URMA/UVB channels to the URMA/UVB channels of the Home-side node.
3. Home-side sysSentry receives the fault handling result via URMA/UVB channels.
4. The Home-side node proceeds with the kernel fault handling operation and continues rebooting.
5. If the Home-side node does not receive the notification from UBPRM within a specified time, it also reboots.

Appendix A Abbreviations

Abbreviations	Full Name
NUMA	Non-Uniform Memory Access
OBMM	Ownership-Based Memory Management
UB	UnifiedBus
UBComm	UB Communication
UBFM	UB Fabric Manager
UBMM	UB Memory Management
UBPRM	UB Pooled Resource Manager
UBPU	UB Processing Unit
UBRT	UB Root Table
UBS Engine	UB Service Core Engine
UDMA	UB Direct Memory Access
UMDK	UB Memory Development Kit
UMMU	UB Memory Management Unit
UMS	UB Memory-based Socket
URMA	Unified Remote Memory Access
URPC	Unified Remote Procedure Call
UVB	Unified Virtual Bus

Note: For other terms, see Section 1.3.